

PY1617: Foundations of R for Statistics and Data Science

Component Notebook

Dr Abigail Page & Dr James Winters

Contents

1	Introduction to R: Week 1	5
1.1	Overview of the R Component	5
1.2	Aims of the Component	5
1.3	Learning Objectives	6
1.4	What Is Data Science?	6
1.4.1	The data science workflow	6
1.5	What Is R?	8
1.6	What Is RStudio?	8
1.6.1	Why we use R & RStudio	9
1.7	Challenges of Learning R	9
1.8	Installing R and RStudio	10
1.8.1	Installing R	10
1.8.2	Installing RStudio	10
1.9	First Steps in RStudio	10
1.9.1	Console	10
1.9.2	Environment and history	10
1.9.3	Files, plots, packages, helps and viewer	11
1.9.4	R scripts	11
1.10	Basic R Expressions	11
1.10.1	Writing in the console	11
1.10.2	Writing in a script	12
1.11	Working with R: Key things to know	13
1.11.1	R is literal	13
1.11.2	R ignores some spacing	13
1.11.3	R can sometimes detect incomplete commands	14

2	Workflow in R: Week 2	14
2.1	Organising folders on your computer	14
2.2	Object based computing	15
2.2.1	What is an object?	15
2.2.2	Why use objects?	16
2.2.3	Object Names	16
2.2.4	Reusing objects	17
2.2.5	Activity 2.1: Working with objects	17
2.3	Understanding the R environment	18
2.3.1	What lives in the environment?	18
2.3.2	The Environment pane	19
2.3.3	Sessions and memory	19
2.4	RStudio projects	19
2.4.1	Why use projects?	20
2.4.2	Activity 2.2: Creating a project for this component	20
2.5	Working directory	20
2.5.1	Checking your working directory	20
2.6	Saving scripts and naming files	22
2.7	Reading and writing files using relative paths	23
2.7.1	Reading in datasets	23
2.7.2	Writing data	24
2.8	Functions and Arguments	24
2.8.1	Functions	24
2.8.2	Arguments	25
2.9	Packages	25
2.9.1	Installing packages	26
2.9.2	Loading packages	26
3	Data in R: Week 3	26
3.1	R Scripts in more detail	27
3.1.1	Structuring your script from the very top	27
3.1.2	Start with packages and data	27
3.1.3	Using # and ## for the RStudio document outline	28
3.1.4	Activity 3.1: Create a new R script for this weeks material and structure it as follows (copying the code section about for the first three sections:	29
3.2	Data types	29
3.2.1	Common data types in R	31

3.2.2	Coercion between types	31
3.3	Vectors	32
3.3.1	Creating vectors	32
3.3.2	Vectors are ordered	33
3.3.3	Indexing vectors	34
3.3.4	Removing values	34
3.3.5	Text vectors and factors	35
3.3.6	Activity 3.2: Making a factor	36
3.3.7	Ordered factors	36
3.4	Data frames	37
3.4.1	Exploring a data frame	37
3.4.2	Exploring with comparison operators	39
3.5	Re-labelling variables	40
3.6	Introducing pipes	41
4	Data Manipulation in R: Week 4	42
4.1	Overview: Why do we need data manipulation?	42
4.1.1	Raw data are rarely analysis-ready	42
4.1.2	What counts as “manipulation”?	42
4.1.3	Manipulation is part of thinking	42
4.1.4	From workflow to analysis	43
4.2	What is tidy and clean data?	43
4.2.1	Tidy data	43
4.2.2	Clean data	43
4.3	The tidyverse: transforming variables and datasets	43
4.3.1	What packages are inside the tidyverse?	44
4.3.2	Thinking in “verbs”	44
4.3.3	<code>filter()</code> – choosing rows in a dataset	45
4.3.4	<code>select()</code> – choosing columns	48
4.3.5	<code>mutate()</code> – creating and recoding variables	50
4.3.6	<code>summarise()</code> – summarising data	55
4.3.7	<code>group_by()</code> - doing things within groups	57
4.3.8	Activity 4.1: using tidyverse verbs	59
4.4	Wide and long data	59
4.4.1	Wide data	59
4.4.2	Long data	60
4.4.3	Converting wide to long with <code>pivot_longer()</code>	60

4.4.4	Pivoting long to wide with <code>pivot_wider()</code>	61
4.4.5	Activity 4.2: pivoting the tidyverse df	62
4.5	Missing data (NAs)	62
4.5.1	Where do missing data come from?	62
4.5.2	Missing values in R	63
4.5.3	Finding missing data in a dataframe	64
4.5.4	Removing missing data	66
4.5.5	Option C – remove NAs only for specific variables	67
4.5.6	Why shouldn't we swap NAs for 0's?	68
5	Data Visualisation in R: Week 5	69
5.1	Summarising data	69
5.2	Plotting with <code>ggplot2</code>	69
5.2.1	Iris flower data frame	69
5.2.2	Exploration of Iris dataset	72
5.2.3	Adding aesthetics	74
5.2.4	Adding layers to your plot	80
5.2.5	Streamlining your code	86
5.2.6	Facetting your plots	88
5.2.7	Changing axis ticks	90
5.2.8	Text on plots	91
5.2.9	Exercises 5.1:	92
6	Conditional Programming: Week 6	94
6.1	Logical values	94
6.2	if and else statements	94
7	Pipes and Loops: Week 7	94
7.1	Pipes (<code>%>%</code>)	94
7.2	for loops	94
7.3	Repeating operations	94
8	Writing Functions from scratch: Week 8	94
8.1	Why write functions?	94
8.2	Function structure	94
8.3	Writing your first function	94

9 Simulating Data: Week 9	94
9.1 Why simulate data?	94
9.2 Generating random data	94
9.3 A simple simulation example	94

1 Introduction to R: Week 1

Welcome to the R component of **PY1617 Employability: Foundations of R for Statistics and Data Science**

This notebook brings together all material for the R component of the module, spanning ten weeks of teaching.

In this first week, the focus is on **orientation**: understanding what R is, why we are using it, how it fits into data science and statistics and getting everything set up correctly on your computer.

1.1 Overview of the R Component

This component runs across **ten teaching weeks**, starting from the very basics of using R and progressing to writing functions and simulating data by the end of the term.

Teaching is shared across the module:

- 1) **Weeks 1–5** are taught by Dr Abigail Page and focus on foundational R skills, data handling, and visualisation
- 2) **Weeks 7–10** are taught by Dr James Winters and focus on programming concepts and simulation

By the end of the component, you will be able to use R confidently as a tool for working with data in academic, research and workplace contexts.

1.2 Aims of the Component

The R component aims to develop **foundational skills in R** that underpin data analysis, statistics and data science.

Specifically, the component aims to: 1) Build confidence in using R and RStudio 2) Develop good habits for writing clear, reproducible code 3) Introduce core data workflows used in statistics and research 4) Prepare you for later statistics modules in Years 1 and 2 5) Develop digital and data literacy skills valued in the workplace

1.3 Learning Objectives

By the end of this component, you should be able to:

- 1) Use **R** and **RStudio** confidently, including scripts, projects and packages
 - 2) Understand and work with **data types, objects and data frames** in R
 - 3) Apply a **tidy data workflow** to clean, transform and reshape data
 - 4) Create and label **new and recoded variables**
 - 5) **Summarise and visualise data** effectively using ggplot2
 - 6) Use basic **programming concepts** (loops and functions) to automate tasks
 - 7) Write **clear, reproducible code** suitable for academic and workplace contexts
 - 8) Use these skills to **simulate data**
-

1.4 What Is Data Science?

Data science is the process of turning data into understanding and insight.

This typically includes:

- 1) Managing data
- 2) Cleaning and preparing data
- 3) Exploring and visualising data
- 4) Modelling and interpreting results
- 5) Communicating findings clearly

Data science is best thought of as a **workflow**, rather than a single technique.

1.4.1 The data science workflow

Throughout this module, we will refer to the **data science cycle (Figure 1.1)**, which describes how data are:

- 1) Imported: bringing data into R
- 2) Prepared and transformed: cleaning the data so you can do what you want with it.
- 3) Explored and visualised: explore patterns and trends

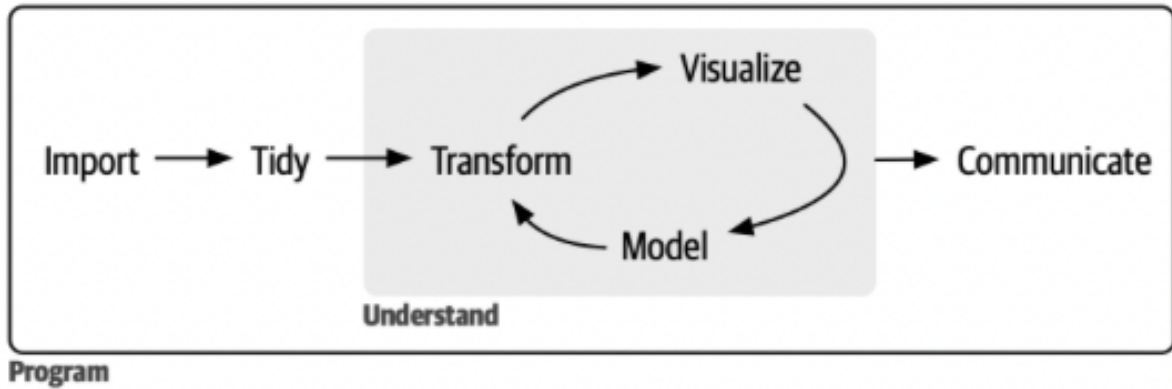


Figure 1: *Figure 1.1 Workflow Diagram from R4DS textbook*

- 4) Modelled: apply statistical models
- 5) Communicated: report your results clearly in plots and tables

We can think of this as a cycle because you don't do the steps only once, but you loop back and forth.

The first step in any data analysis is importing your data into R. This usually involves loading data from a file into a data frame. Without access to your data within R, you cannot perform any analysis. Once your data is imported, it's important to tidy it. Tidy data has a consistent structure where each column represents a variable and each row represents an observation and each cell represents a value (Figure 1.2). This organization makes it easier to work with the data, allowing you to focus on analysis rather than data cleanup.

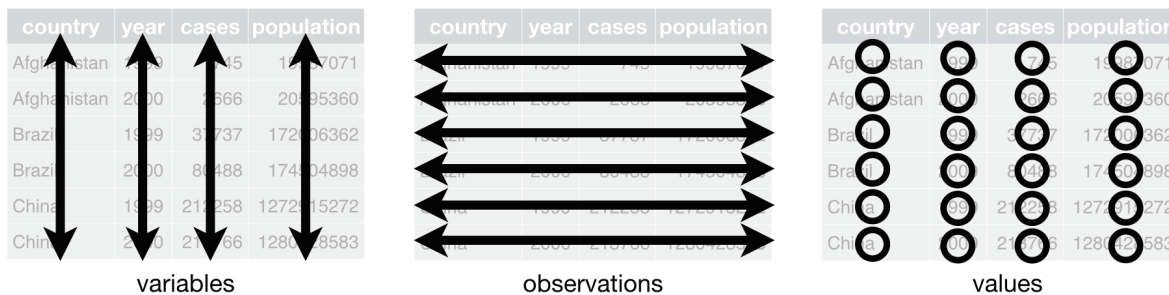


Figure 2: *Figure 1.2 Variables, observations and values from R4DS textbook*

After tidying, the next step is often transforming your data. Transformations can include filtering for specific observations (e.g., data from a particular participant or year), creating new variables from existing ones (e.g., calculating speed from distance and time), or computing summary statistics (e.g., counts, averages). The process of tidying and transforming data is commonly called data wrangling, because getting data into a usable form can be challenging.

With clean and well-structured data, you can begin generating insights through visualisation and modelling. These approaches complement each other:

- Visualisation is a human-centred activity. Well-designed visualisations can reveal unexpected patterns,

suggest new questions or indicate that you may need additional data. While visualisations are invaluable for understanding, they rely on human interpretation and don't scale easily to large datasets.

- Modelling provides a computational approach. Once your questions are clearly defined, models can answer them efficiently and at scale. However, models operate under assumptions and cannot challenge those assumptions on their own. They are powerful, but inherently limited in their ability to surprise.

Finally, communication is essential. The insights you derive from analysis - whether through visualisations, models or summaries — are only useful if you can clearly explain them to others.

Throughout the process, programming supports every step. While you don't need to be a programming expert, stronger programming skills allow you to automate tasks, work more efficiently and tackle new problems more effectively. These tools cover most of what you'll need in a data science project.

This R component focuses primarily on the **foundational stages of this cycle: importing, tidying and visualising data**. These are the skills that everything else depends on. Data modelling and communication will be focused on in other components and modules.

1.5 What Is R?

R is a programming language designed specifically for **data analysis, statistics and graphics**. It was first developed in 1996 and is now widely used in academia, research and industry.

R is:

- A command-driven language
- Highly flexible and extensible
- Widely used for statistical analysis and data visualisation

You may have encountered other statistical software such as SPSS, Stata, or MATLAB. R differs from these in that it is:

- Script-based rather than menu-driven (you write things rather than click things)
- Highly customisable
- Open source and free to use

1.6 What Is RStudio?

RStudio is the software environment we will use to work with R.

You can think of:

- R as the engine which does the work
- RStudio as the dashboard which makes it easy to do the work

RStudio provides:

- A script editor

- A console for running code
- Tools for viewing data, plots and files
- Integrated help and documentation

Once we have installed RStudio we no longer need to directly access R!

1.6.1 Why we use R & RStudio

R & RStudio is widely used because:

- Faster & more accurate than hand calculations: Computers handle repetitive statistical tasks easily. Doing a few calculations by hand helps you understand concepts, but mostly it's about intuition.
- Better than spreadsheets: Excel and similar tools feel familiar but are limited for serious analysis. R is far more flexible and powerful.
- Free & open source: No expensive licenses, no student-version traps—R is professional-level software at no cost.
- Highly extensible: Thousands of additional packages are freely available, letting you use cutting-edge methods and advanced techniques as you grow.
- Learn programming along the way: Using R teaches programming skills, which are essential for modern research, online experiments, automated data collection, AI, and computational methods.
- Widely used in research: Learning R gives you access to standard tools in statistics and psychology, putting you closer to advanced, real-world methods.

Bottom line: R can be challenging at first, but its power, flexibility, and relevance make it the best choice for serious data analysis.

1.7 Challenges of Learning R

Learning R can feel challenging at first, especially if you do not have a programming background.

Common difficulties include:

- A steep initial learning curve
- Small errors in code causing unexpected problems (e.g. a missing closing bracket or misplaced comma can be frustrating)
- Not knowing how to phrase questions when searching for help

This is normal.

A useful rule of thumb:

Knowing what to search for is a large part of working effectively in R. AI tools make this a lot easier!

Over time, you will build familiarity with common patterns and solutions.

1.8 Installing R and RStudio

Before you can start using R, you need to install **both R and RStudio**.

1.8.1 Installing R

1. Go to: <https://cran.r-project.org>
2. Select your operating system (Windows, macOS, or Linux)
3. Download and install the **latest version** of R

Always install R **before** installing RStudio.

1.8.2 Installing RStudio

1. Go to: <https://www.rstudio.com/products/rstudio/download/>
2. Download **RStudio Desktop (free version)**
3. Install the latest version for your operating system

Once installed, you will **open RStudio**, not R itself.

Opening RStudio automatically starts R in the background.

Finally you will need to download Rtools from Cran: <https://cran.r-project.org/bin/windows/Rtools/>. > Make sure this is installed before moving on.

1.9 First Steps in RStudio

When you open RStudio for the first time, you will see several **panes** (also called panels). Each has a different purpose to help you work efficiently in R.

- The **console** (where code runs) on the left
- The **environment** (where objects are stored) on the top right
- Panels for plots, files, and help on the bottom right

1.9.1 Console

The **console** is where you **type and run R commands directly**. We will play around with this at the end of the session. The console executes your commands immediately and shows the results.

1.9.2 Environment and history

The Environment tab shows all the objects you have loaded into R or created (variables, data frames, lists, etc). The History tab keeps a record of all the commands you have run in the console. You can click on it and see what you have just done, and re-run it.

1.9.3 Files, plots, packages, helps and viewer

This is where you can view lots of useful pieces of information and access your files:

- 1) Files: Lets you browse files in your project folder. You can import data from here.
- 2) Plots: Displays graphs and visualisations you create. This pops up automatically when you make a plot.
- 3) Packages: Displays which R packages are installed and loaded. What you can use to install more packages (we will get to this)
- 4) Help: Shows documentation for functions you search.
- 5) Viewer: Used for HTML outputs and interactive visualisations.

These tabs help you navigate your project, visualize results and find help quickly. We will come to each of these sections in turn.

First, there is a final section of RStudio which is only viable when you open up a R Script.

1.9.4 R scripts

An R script is a file where you can write, save and edit multiple lines of R code. Like a word document for R code! Scripts have the file extension .R and are saved in a folder on your computer.

You can either open a blank script: - Go to File (top left piece of paper with a green plus) - New File - R Script

This creates a script where you can write and save your code.

OR you can load a pre-made R script which you or someone else have made to run. You can download the pre-made R Script for Week 1 session here:

Download the R Script

1.10 Basic R Expressions

You can:

- Type directly into the **console**, or
- Write code in a **script**, which can be **saved, shared and reproduced**.

Tip: If you write directly in the console, your work is **not saved** when RStudio closes.

So while we will show you how to write script directly into the console today, in future we will be working with scripts.

1.10.1 Writing in the console

In R, `<-` is the **assignment operator** (not `=`).

- Example: `x <- 500` tells R “x equals 500.”

- Whenever you type `x`, R will read it as 500.

Steps:

- 1) In the console (after the `>`), type:
- 2) `"x <- 500"`
- 3) press enter
- 4) write `x` in the next line
- 5) press enter

it will look like this:

```
x <- 500
```

Question: when you select `"x"` and press enter what is the output?

Activity: practice with different numbers and letters

In the top right corner of the console you will see a **brush** icon. If you click on it, it clears your console environment. Do this now.

You now have a blank screen so have lost your work. If you go to the history tab now you can see your previous code, and doubling clicking it will bring it back to the Console for you to re-run!

But because you cannot save your work in the console we recommend you always write code in an R script.

1.10.2 Writing in a script

Lines starting with `#` are **comments**. These are notes you leave for yourself or others to help understand what it is you have done

- R **ignores them**, so they are not run.
- They can say anything you like
- They are essential - you will thank yourself later for a well commented script!

Below is what working in a script looks like.

```
# THIS IS A COMMENT!

# To run the code below just select the lines and press 'run' above
 #(or control/command + enter). You can run line by line, or select it all at once.

x <- 10 # telling R that x now equals 10
print(x) # printing this in the console
```

```
## [1] 10
```

```
# so this has done the same as above, but now your work is saved here.
```

```
y <- 10 + 20 # telling R that y now equals 10 + 20  
print(y) # printing this in the console
```

```
## [1] 30
```

```
z <- "hello" # telling R that z now equals "hello"  
print(z) # printing this in the console
```

```
## [1] "hello"
```

```
# you can also get R to print by highlighting the text and press  
# enter/run (called auto-printing) which is often easier to do!
```

```
a <- "hello"  
b<- "world"  
a
```

```
## [1] "hello"
```

```
b # run all four lines at once
```

```
## [1] "world"
```

1.11 Working with R: Key things to know

Some final points about R's behaviour, errors, spacing and “unfinished commands”

1.11.1 R is literal

R does exactly what you tell it. It has no intuition, no autocorrect and no ability to guess your intent. Typos matter. For example $10 = 20$ gives you an error

```
10 - 20 # produces -10, but may not be what you meant
```

```
## [1] -10
```

Even simple mistakes can produce wrong answers without errors. Always double-check what you type.

1.11.2 R ignores some spacing

Extra spacing usually doesn't matter

```
10 + 20
```

```
## [1] 30
```

```
10+20      # both work fine
```

```
## [1] 30
```

- but don't split words or function names (e.g. you can't say `citat ion()`)
- consistent formatting is good practice

1.11.3 R can sometimes detect incomplete commands

- If R thinks a command isn't finished, it waits for more input
- This also works for long commands broken over multiple lines
- To cancel a command mid-typing, press escape to return to the prompt.

R is a powerful tool, but it's precise and literal. Be careful with typos, spacing, and parentheses, and remember that errors are part of learning. With practice, reading and understanding R's responses becomes easier.

2 Workflow in R: Week 2

This week focuses on good workflow habits when working with R. These skills may feel minor now, but they are essential for reproducible analysis, collaboration and avoiding frustration later in the course.

Using this notebook: In the lecture you will work through this notebook. You should copy the code segments below into a new R script (instructions given below) to practice and work through the exercises and questions. To *make a new script in R*: select the paper icon with a green plus on it in the top-left corner and select 'R Script' from the top of the drop-down menu. You now have a blank page in front of you. You will make a new script each week.

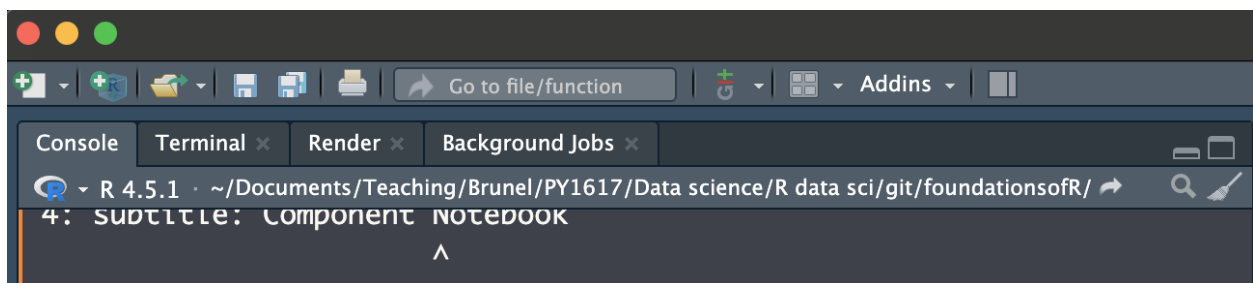


Figure 3: *Figure 2.1 To create new script: click the sheet with the green plus button.*

2.1 Organising folders on your computer

Before opening RStudio, it is important to think about how your files are organised on your computer. Much confusion and many errors in data analysis come from poorly organised files rather than problems with the code itself.

Some guidance on best practice:

- Create a main course folder (e.g. PY1617) and keep all related work inside it

- Inside this folder, create sub-folders for different types of files, such as:
 - **data** – raw and cleaned data files
 - **scripts** – R scripts you write
 - **outputs** – plots, tables or processed data created by R
- Use clear, informative names that describe what the file contains
- Avoid spaces and special characters; use hyphens - or underscores _

Good organisation makes it easier to:

- Find files quickly
- Understand your work weeks or months later
- Share work with others
- Avoid breaking code that relies on file paths

Exercise 2.1 Create a folder structure for this module on your computer.

Call the first folder “PY1719”, with sub-folders for “analysis”, “methods” and “skills”. In the **skills folder** include the following folders: 1) Data 2) Scripts 3) Outputs

2.2 Object based computing

R is an **object-based programming language**. This means that most of what you do in R involves **creating objects** and then **doing things to those objects**. It is useful to remember that everything in R is an object.

An **object** is simply a way of storing information so that R can use it later.

2.2.1 What is an object?

An object can store:

- A single number
- Text (a character string, e.g. “Hello”)
- A collection of values (e.g. 1, 5, 7, 3)
- An entire dataset (rows and columns)

For example, last week we learnt how to store a single number within the ‘x’ object

```
# make x = 10
x <- 10

# print x
x
```

```
## [1] 10
```

Here:

- `x` is the object name
- `<-` assigns a value
- 10 is the value stored inside the object

Once an object is created, you can reuse it:

```
# add 5 to x
x + 5
```

```
## [1] 15
```

R remembers what `x` refers to and uses its stored value to add 5 to 10 = 15

2.2.2 Why use objects?

Objects allow you to:

- Avoid repeating the same values
- Work with complex data (such as datasets)
- Write clearer, more readable code
- Build analyses step by step

For example, instead of repeatedly typing a dataset name or value, you store it once and refer to it when needed.

2.2.3 Object Names

Object names should be:

- Short but meaningful
- Written in lowercase (as a rule you don't want to be messing around with caps in your code)
- Use underscores rather than spaces

Examples of good names:

- `age`
- `survey_data`
- `mean_score`

Examples to avoid:

- `x1` (often the default at importing, so quickly confusing)
- `my stuff` (too generic)
- `FINALdata` (you will end up with `finalfinalFINALdata` which is confusing)

Good naming makes your code easier to understand and reduces mistakes.

2.2.4 Reusing objects

Once an object exists, it can be:

- Used in calculations
- Passed into functions
- Combined with other objects

```
# y = x*2
y <- x * 2 # Here, y is created multiplying the value stored in x by 2

# print y
y
```

```
## [1] 20
```

NB - if you don't save the output as a new object, R prints it automatically. If you save it as a new object (like `y` here) then you will need to ask R to print it to view and confirm.

Object-based thinking is central to working effectively in R: create objects, then operate on them.

2.2.5 Activity 2.1: Working with objects

This activity will help you practise creating and using **objects** in R.

Step 1: Create the following objects in your script under a commented header `# Activity 2.1` by copying this code into your script

```
# Activity 2.1

age <- 20
years <- 5
name <- "Alex"
score <- 68
```

Question 2.1: Check that these objects appear in the Environment pane. Can you see the correct values attributed to the objects?

Step 2: Create a new object using the objects you already made:

```
# add 1 to age
age_next_year <- age + 1
```

Question 2.2: What is Alex's age after five years?

Step 3: R allows you to overwrite objects, so update the age variable

```
# overwrite age with 25
age <- 25
```

Question 2.3: what happened to the original value of age?

Be careful: once an object is overwritten, the old value is lost unless you saved it elsewhere.

Step 4: make a simple single observation dataset to add to your environment. First, check that all your objects already made appear in your environment pane (name, age, years and score). Now combine these objects into a dataset with one row. The dataset is called 'student':

```
# make a dataframe called 'student'
student <- data.frame( # function is data.frame()
  name = name, # includes name called the same as before
  age = age, # age
  score = score # score
)

# name = name means bring the 'name' variable across and call it 'name'
# in the new dataframe.
```

You have created a dataset with one observation!

Reflection 1) Which objects currently exist in your Environment? 2) What happens if you restart RStudio and type age again? 3) Why does this show the importance of scripts?

2.3 Understanding the R environment

When you work in R, everything you create or load during a session lives in the R environment. **The environment is R's internal workspace, where objects such as data files, variables are stored and easy to access.** This becomes very useful the more complicated your analysis.

2.3.1 What lives in the environment?

- Data you read in from files (e.g. CSVs)
- Variables you create (numbers, text, data frames)
- Objects created by running code in scripts or the console (like we just did)

For example:

```
# make x = 10 and y = 100
x <- 10
y <- 100
```

Both x and y are now alongside your 'student' data and other objects now exist in the environment and can be used by other lines of code.

You can check what objects are in your environment with the `objects()` function

```
# check the objects in your environment
objects()
```

```
## [1] "a"          "age"         "age_next_year" "b"
## [5] "name"       "score"       "student"       "x"
## [9] "y"         "years"      "z"
```

You can remove objects from your environment as well as adding them. In RStudio, the easiest way to remove variables is using the Environment pane.

- click the brush at the middle of the environment pane
- This removes all the objects in your environment, resetting it.

Deleting variables cannot be undone. Once a variable is removed, it no longer exists in the environment. You can also remove variables using code, which is useful if you want to be explicit in a script. You do this with the `rm()` function:

```
# remove x and y variables
rm(x, y)
```

```
# re-check
objects()
```

```
## [1] "a"           "age"          "age_next_year" "b"
## [5] "name"        "score"        "student"       "years"
## [9] "z"
```

2.3.2 The Environment pane

The Environment pane shows:

- A list of all current objects
- Their names and basic information (e.g. size or type)

This pane helps you:

- Check whether your data has loaded correctly
- See what objects are available to use
- Avoid re creating objects unnecessarily

If something is not in the Environment, R cannot use it.

2.3.3 Sessions and memory

- The environment only exists while RStudio is open
- When you close RStudio, the environment is cleared
- This is why saving scripts is essential: scripts allow you to recreate the environment by rerunning code
- We can also avoid this by using projects...

2.4 RStudio projects

An RStudio Project connects RStudio to a specific folder on your computer. Keeping all the files associated with a given project (input data, R scripts, analytical results and figures) together in one directory is such a common practice that RStudio has built-in support for this via projects.

2.4.1 Why use projects?

- Automatically sets the correct working directory
- Keeps scripts, data, and outputs together
- Makes it easier to return to your work later

2.4.2 Activity 2.2: Creating a project for this component

We are now going to make use of your folder structure you made earlier to save our R project in an easy to find location. Please follow these steps:

1. Open RStudio
2. File → New Project
3. In browse select your skills component folder you made earlier
4. Call your project ‘foundations’

NB - R saves the project in its own folder, you can delete this and just copy and paste the ‘foundations’ project into your ‘Skills’ folder.

Now, quit RStudio. Inspect the folder associated with your project — notice the `.Rproj` file. Double-click that file to re-open the project. Notice you get back to where you left off: it’s the same working directory and command history, and all the files you were working on are still open.

2.5 Working directory

Once you have made your project you will have a ‘home’ to save all your code and output. This is called the working directory is the folder R looks in straight when reading or saving files. This is like in your computer when you first go to save a word document it first suggests ‘your documents’ folder.

Basically, now **R will default to save everything, and import everything from your project folder you just made!** This is super useful to be easily able to find your work.

2.5.1 Checking your working directory

RStudio shows your current working directory at the top of the console:

And you can print this out in R code by running `getwd()`

```
# check working directory
getwd()
```

```
## [1] "/Users/abigailpage/Documents/Teaching/Brunel/PY1617/Data science/R data sci/git/foundationsofR"
```

If you want to know the associated list of files and/or sub-directories within your working directory you can use `dir()`

```
# check what's in your WD
dir()
```

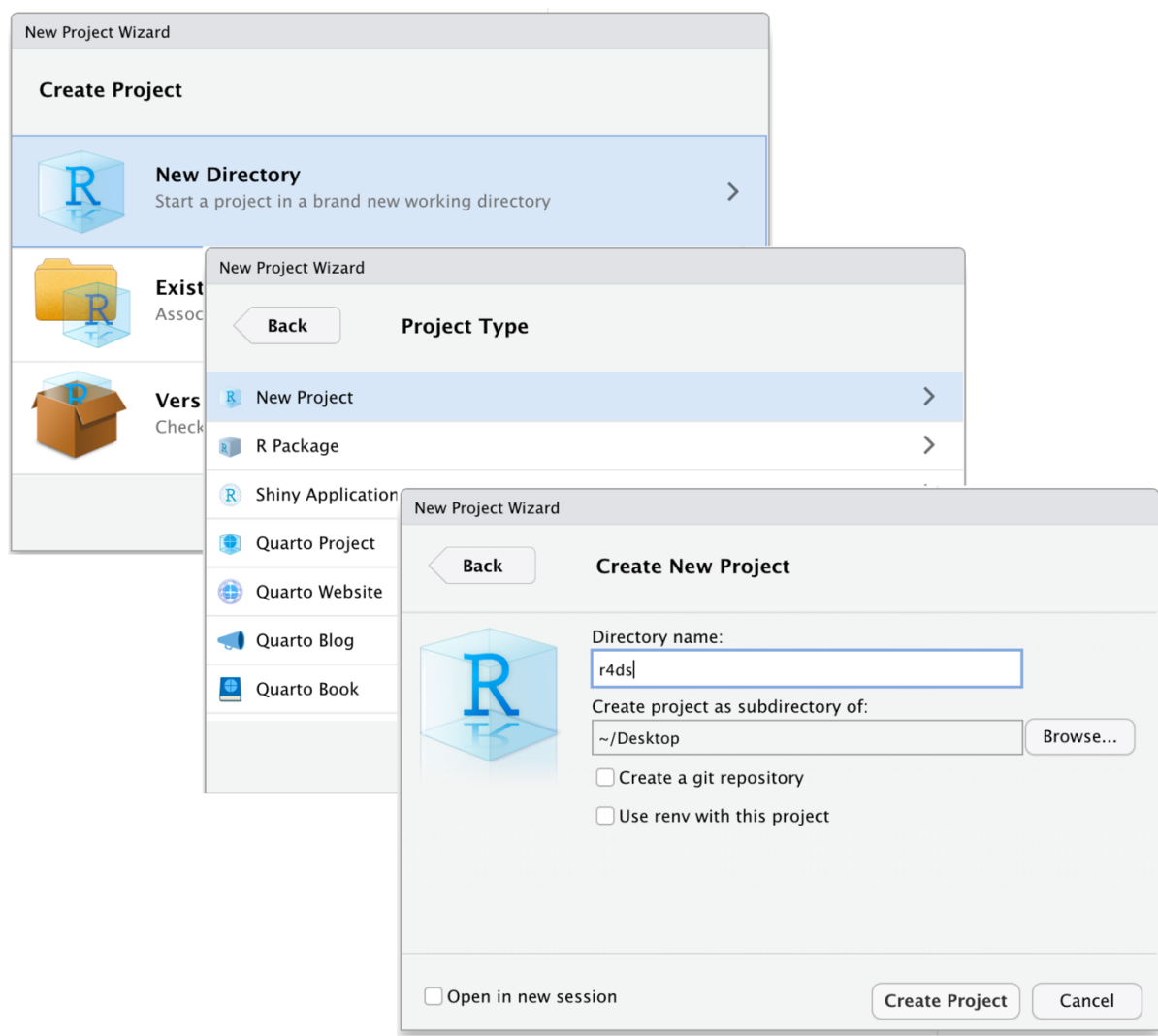


Figure 4: *Figure 2.2 To create new project: (top) first click New Directory, then (middle) click New Project, then (bottom) fill in the directory (project) name, choose a good subdirectory for its home and click Create Project. From R4DS.*

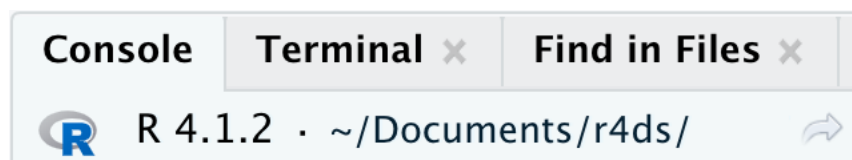


Figure 5: *Figure 2.3 Image of your working directory location at the top of your console. From R4DS.*

```
## [1] "brunel_logo.png"          "data"
## [3] "dev_data.csv"             "example script.R"
## [5] "foundationsnotebook.html" "foundationsnotebook.log"
## [7] "foundationsnotebook.nb.html" "foundationsnotebook.Rmd"
## [9] "foundationsnotebook.tex"   "foundationsofR.Rproj"
## [11] "foundationsposter.html"    "foundationsposter.pdf"
## [13] "foundationsposter.Rmd"     "header.png"
## [15] "linetypes.png"            "newscrip.png"
## [17] "outputs"                  "projects.png"
## [19] "saveaspdf.R"              "shapetypes.png"
## [21] "STUDENTTEMPLATE.Rmd"      "subheadings.png"
## [23] "tidy-1.png"               "week4df.R"
## [25] "workflow.png"             "workingdir.png"
```

2.6 Saving scripts and naming files

RStudio automatically saves the contents of the script editor when you quit, and automatically reloads it when you re-open. Nevertheless, it's a good idea to avoid `Untitled1`, `Untitled2`, `Untitled3`, and so on and instead save your scripts and to give them informative names.

Script files are those with a `.R` file extension. These aren't data files at all; rather, they're used to save a collection of commands that you want R to execute later.

It might be tempting to name your files `code.R` or `myscript.R`, but you should think a bit harder before choosing a name for your file. Three important principles for file naming are as follows:

Good file names should:

- Be machine readable: avoid spaces, punctuation, and special characters
- Be human readable: the name should describe what the file does or contains
- Sort in a sensible order: numbering files (e.g. 01-, 02-) helps keep work organised

For example, instead of:

- `code.R`
- `final.R`
- `temp.R`

Prefer:

- `01-load-data.R`
- `02-exploratory-analysis.R`
- `03-clean-data.R`

Clear file names reduce confusion and help others (and your future self) understand your workflow.

Exercise 2.2: At the moment you should have a unnamed script you have been using for this weeks content. Please now give it a sensible name (e.g. `week2`) when you save it. You can save it by pressing the disk icon in the script pane in the top left. This will automatically open up the interface to your working directory.

2.7 Reading and writing files using relative paths

Once you are working inside an RStudio Project, you should always use relative paths, not absolute paths.

A relative path is written relative to the project's working directory. For example: `data/diamonds.csv`

This is telling R to access the data file (.csv) diamonds in the 'data' folder which is saved in your R project folder (set by your working directory).

This works on any computer, as long as the file is inside the project folder. This makes your code portable and easy to share (doesn't have the long file directory with your user name etc. which won't work on other computers)

An absolute path points to a specific location on one computer, such as:

- Windows: `C:/Users/...`
- Mac/Linux: `/Users/...`

You should avoid absolute paths in scripts because no one else will have the same folder structure. This shows you why its so important to have a good folder structure at the start of any project.

One final detail:

- Windows uses backslashes (`\`) in file paths
- Mac and Linux use forward slashes (`/`)

R can handle both, but backslashes need special treatment and can cause errors. For this reason, always use forward slashes (`/`) in R, even on Windows.

2.7.1 Reading in datasets

So now we know all this we can import data from our data sub-folder into R easily. Comma separated value (CSV) files are those with a .csv file extension. These are just regular old text files, and they can be opened with almost any software. It's quite typical for people to store data in CSV files, precisely because they're so simple.

We will be using CSV files in this component as a rule.

```
# import (read) a dataframe
mydata <- read.csv("data/mydata.csv") # note that if you called your folder 'Data'
# you will need to use the capitalised Data, not my 'data'.

# you are telling r what to call the # dataset (mydata) and you are telling R
# where to find the data (in the data folder, called mydata.csv)
```

We can also ask R for some specifics when we are reading in our data. These specific requests are called 'arguments' in R terminology:

- The argument **header** is a logical variable indicating whether variable names are included in the top of each column or not.

```
# read with header
mydata <- read.csv("data/mydata.csv", header = TRUE)
```

We inspect the data using the function `head()` which presents the first 6 rows of the dataset.

```
# check the first 6 rows  
head(mydata) # it's a single variable of height
```

```
##   height  
## 1    1.3  
## 2    1.3  
## 3    1.7  
## 4    1.8  
## 5    1.4  
## 6    1.6
```

2.7.2 Writing data

‘Writing’ data refers to when we want to export data from R on to our computer. You could otherwise call it saving, like you would in excel. The standard function in R to export data is `write.csv()`. Let’s try this with a new dataset. We will now use one of the built in data-sets that R has, called ‘iris’.

R has a couple of in-built learning datasets, so we are just going to save this to your computer. We are going to use the relative path so we’re sending it to the outputs folder within your working directory. This just means you know where to find it.

```
# save iris to your 'outputs' folder  
write.csv(iris, "outputs/iris.csv")  
  
# here we're saying, find the file called  
# 'iris' then write it to csv. in the outputs folder within the working  
# directory. Go check this folder now to confirm.
```

Exercise 2.3: bring together all your skills now and make a dataset with the following objects
1) your name, 2) your age, 3) your fav colour and 4) your height. The save this as “mydf.csv” in your outputs folder. Then go to this folder and open it in excel.

2.8 Functions and Arguments

Just before we wrap up this week we are going to introduce formally a last few key things in R: functions, arguments and packages.

R works by creating objects, applying functions to objects to make them doing specific things and controlling behaviour using arguments.

2.8.1 Functions

Functions perform actions on objects. We saw this actually just above with the `read.csv()` which is the function, and the argument of `header=TRUE`. To give another example using the data we just read into R.

```
# the mydata is just one variable - 'height' so lets compute the mean for this  
mean(mydata$height)
```

```
## [1] NA
```

```
# we will learn more about dataframes next week, but the  
# way I have called the object has changed slightly to mydata$height  
# this is because height is a variable within the dataset, so i'm asking R to  
# look into the object dataset [mydata] to find the object variable [height]  
# but we will come back to this
```

This function here is `mean()` and it calculates the average of the values stored in `x`. Rather than calculating the sum of your numbers and dividing manually every time, you can use the mean function, which performs this calculation for you with a single command. This makes your work easier and more efficient. Later in the module, you'll learn how to write your own functions to perform custom calculations automatically.

However, here the function to compute the 'mean' hasn't worked because it has returned the result `NA`. This is because R cannot compute a mean if there are `NA`s in the dataset (which there is if you look at the `mydata` tab in the viewer) and you haven't told R how to handle these `NA`s. You need an **argument** to do this.

2.8.2 Arguments

After the functions comes the **arguments** - the content of the brackets. Arguments change how a function behaves:

```
# add the na.rm argument  
mean(mydata$height, na.rm = TRUE)
```

```
## [1] 1.59
```

```
# na.rm means 'remove NAs' so your telling R  
# just to compute the mean ignoring the NAs.
```

So here:

- `mydata$height` is the object
- `mean()` is the function
- `na.rm = TRUE` is an argument telling R how to handle missing values (telling R to remove all missing values when you are calculating the mean)

Understanding this pattern — **objects** → **functions** → **arguments** — is key to reading and writing R code.

2.9 Packages

A package is basically just a big collection of functions, data sets and other R objects that are all grouped together under a common name. Some packages are already installed when you put R on your computer, but the vast majority of them of R packages are out there on the internet, waiting for you to download, install and use them.

R packages are basically add-ons to base R. Base R does a number of simple functions, but thanks to the open-source ethos behind R the R community has built a large number of packages (over 14,000 on the CRAN where you downloaded R from) to solve common problems or offer new analysis techniques. This is the reason why R is so great. A package brings new functionality in the form of data, code, documentation and tests.

Generally, when you open up R, only a few of these packages (about 7 or 8) are actually loaded. Basically what it boils down to is this:

- A package must be installed before it can be loaded.
- A package must be loaded before it can be used.

2.9.1 Installing packages

You only need to install packages once. To so you need to use the install packages command `install.packages("tidyverse")`

This is the R command that does all the work. R then goes off to the internet, has a conversation with CRAN, downloads some stuff, and installs it on your computer. Here we are downloading the tidyverse package which has many of the functions we will be using in the coming weeks.

2.9.2 Loading packages

Loading makes it available for use. You will need to do this every session, unless you are saving your environment. Most scripts you will see start with the required packages at the top, in addition to any required data (we will review scripts set-up in more detail next week). **It is good practice to specify which packages your project requires at the start of any script** as they are unloaded when you re-start R and chunks of code won't work if you don't re-install them (and its way too easy to forget which chunk uses which package).

```
# install tidyverse once downloaded
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.5.2
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.1      v stringr   1.5.2
## v ggplot2    4.0.1      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

The tidyverse is actually a group of core packages. You can see them listed above. We will be making use of dplyr, ggplot2 and readr in the coming weeks.

3 Data in R: Week 3

This week builds on the workflow foundations from Week 2. We will focus on **how to write good scripts** and introduce the **core data structures** used in R. These skills are central to working efficiently, reproducibly and confidently with real data.

3.1 R Scripts in more detail

Before working with data, it is important to develop good coding habits. These habits will save you time, reduce errors and make your work understandable to others.

We started using a script last week but this script had no formatting. You just copied and pasted code from this notebook into it. What this means is that it can be hard to come back to the document and work out what meant what, or where you left off. We have to add structure to our script, just like we have titles and headers in a word document to make it easier to understand.

3.1.1 Structuring your script from the very top

Every script you write should **start with a clear header**. This makes it immediately obvious what the file is for and when it was last worked on.

A good script header usually includes:

- A short title describing the purpose of the script
- Your name and any co-authors
- The date (or date range) of work

```
# =====  
# Script: week3_practice.R -----  
# Purpose: Working with scripts and data frames  
# Author: Your Name  
# Date: 2026-02-01  
# =====
```

This header does not affect how the code runs, but it is extremely valuable for organisation and reproducibility. It will always sit at the top of your script.

3.1.2 Start with packages and data

The first functional section of your script should deal with **anything that must be available before the rest of the code can run**. This usually includes:

- R packages that need to be loaded (assuming they are installed)
- Data files that need to be read into R

Placing these steps at the top means that:

- Errors appear early, not halfway through your analysis
- Anyone running your script knows exactly what they need

A typical structure looks like this:

```
# =====  
# Script: week3_practice.R -----  
# Purpose: Working with scripts and data frames  
# Author: Your Name  
# Date: 2026-02-01  
# =====
```

```

# 1.0 Setup -----

## 1.1 Packages -----
library(tidyverse)

## 1.2 Data import -----
iris_data <- iris # if working from a pre installed dataset.
mydata <- read.csv("data/mydata.csv") # importing from your project

### 1.2.1 Data inspection -----
head(mydata)

## height
## 1 1.3
## 2 1.3
## 3 1.7
## 4 1.8
## 5 1.4
## 6 1.6

```

Note that I have started numbering the sections, again this is just good practice for when you start having really long processing and analysis scripts to help you work out where you are.

3.1.3 Using # and ## for the RStudio document outline

RStudio provides a document outline / drop-down menu (at the bottom right of the script pane) that allows you to jump between sections of a script. This becomes increasingly useful the longer your script gets. You can click on it to quickly move between different sections - kind of like a table of contents.

RStudio recognises comment-based headers using hashes and dashes:

```

# creates a top-level section -----
## creates a subsection -----
### creates a sub-subsection -----

```

For main section headings you use four dashes after a single #. If you use two ## or ### they become nested subheadings underneath the main heading.

Example:

```

# 2. Data import -----
## 2.1 Reading files -----
### 2.1.1 Inspecting the data -----

```

These headers will appear in the RStudio drop-down menu, allowing you to quickly navigate large scripts.

Good practice: Use top-level headers (#) sparingly for major stages of your workflow, and sub-headers (##, ###) for detail.

If you use separators without text you will get untitled in the drop-down menu which helps you navigate the script quickly. e.g.

```
# -----
```

RStudio sees this as a section with no name, so it shows up as “”.

3.1.4 Activity 3.1: Create a new R script for this weeks material and structure it as follows (copying the code section about for the first three sections:

- A header with title and date with 1) set up for packages and data
- Add in the following structure:
 - 2) Data types
 - 3) Vectors
 - 4) Data frames
 - 5) re-labelling variables
 - 6) Pipes

We will populate this structure with sub-headings as we work through the rest of the workbook. As will last week, copy the code chunks into the script. Eg here:

```
# =====
# Script: week3_practice.R -----
# Purpose: Working with scripts and data frames
# Author: Your Name
# Date: 2026-02-01
# =====

# 1.0 Setup -----

## 1.1 Packages -----
library(tidyverse)

## 1.2 Data import -----
iris_data <- iris #from a preinstalled one
mydata <- read.csv("data/mydata.csv")

### 1.2.1 Data inspection -----
head(mydata)
```

Figure 6: **Figure 3.1 The header and set-up.**

Finally make sure you save it to a folder in your R project, with a sensible name.

3.2 Data types

In R, every object has a type (also called its class). The type determines what kind of values the object can store and what operations make sense.

```

# 2.0 Data types -----

## 2.1 common data types -----

# make objects of different types
char_example <- "statistics"
num_example <- 3.14
int_example <- 10
log_example <- TRUE

# check classes
class(char_example)
class(num_example)
class(int_example) # telling you it's a numeric value, which is wrong!
class(log_example)

### Exercise 3.1 - making objects

# make objects |
names <- "Alex"
score <- 72.65
age <- 30
passed <- TRUE

# 3.0 Vectors -----

# 4.0 Data frames -----

# 5.0 Re-labelling variables -----

# 6.0 Pipes -----

```

Figure 7: Figure 3.2 The subheadings starting to be filled up as you work through the next section.

3.2.1 Common data types in R

The most frequently used data types are:

- Character: text values, e.g. names or labels: “a”, “hello”
- Numeric: real numbers with decimals: 2.1, pi
- Integer: whole numbers: 1, 2, 3 etc.
- Logical: TRUE or FALSE values
- Factor: categorical variables with fixed levels

let’s make some now:

```
# Make the four objects of different types
char_example <- "statistics"
num_example <- 3.14
int_example <- 10
log_example <- TRUE
```

Now we can check what data type each of these objects are by using the ‘class’ function which calls the objects we just made in the brackets:

```
# check the class of objects
class(char_example)
```

```
## [1] "character"
```

```
class(num_example)
```

```
## [1] "numeric"
```

```
class(int_example) # telling you it’s a numeric value, which is wrong!
```

```
## [1] "numeric"
```

```
class(log_example)
```

```
## [1] "logical"
```

Although numeric and integer values often behave similarly, they are stored differently in memory. Factors will be discussed in more detail later once we have covered vectors, as they are especially important for statistical modelling.

Exercise 3.1: create four variables of the correct types for the following variable names: name[“Alex”], score[“72.65”], age [30] and passed [TRUE/FALSE]

Exercise 3.2: using the class() function to check the data type. Are any wrong ? Comment in your script using # if any are wrong.

3.2.2 Coercion between types

You can coerce values into different types. R automatically reads an integer (whole number) as numeric (as we saw above!). So you have to tell it. You can either add an L at the end or change its class:

```
# Using L
int_example1 <- 10L
class(int_example1) # r now knows this is a integer
```

```
## [1] "integer"
```

```
# Forcing it by changing the class
int_example2 <- as.integer(int_example)
class(int_example2)
```

```
## [1] "integer"
```

```
# note that I have used the object assignment operator here.
# If I didn't R wouldn't save your changes
```

```
# coerce it back to numeric
int_example <- as.numeric(int_example)
# check the class
class(int_example)
```

```
## [1] "numeric"
```

You can do the same with factors with `as.factor` (but we're coming back to this)

Exercise 3.3: use `as.integer()` function to coerce “age” into an integer and save the result. Re-check it using `class()`.

3.3 Vectors

Vectors are the fundamental data structure in R. A vector stores multiple values of the same type in a single object. For instance, a numeric vector comprises of a list of numbers (e.g. `[1, 2, 6, 6, 3, 6, 2]` is a vector of integers) . In this section we will examine various ways to construct and subset a numeric vector.

3.3.1 Creating vectors

The simplest way to create a vector is with the `c()` (combine) function.

```
# make 2 vectors
vec1 <- c(1, 2, 3, 4, 5)
vec2 <- c(6, 7, 8, 9, 10)

# print the vectors
vec1
```

```
## [1] 1 2 3 4 5
```

```
vec2
```

```
## [1] 6 7 8 9 10
```

Two vectors can also be joined into the same vector using the `c()` command. The two smaller vectors will appear in the new one in the order you specify:

```
# Combine vectors
vec3 <- c(vec1, vec2)
vec3
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

NB - this is different to ADDING vectors, as we will see below!

3.3.2 Vectors are ordered

Vectors are ordered, meaning the position of each element matters. The first element is position 1, the second is position 2, and so on. When you reference a vector by position, you get exactly the element in that position.

```
# make the vector and print
my_vector <- c(10, 20, 30, 40)
my_vector
```

```
## [1] 10 20 30 40
```

The second element in this vector is the number 20, the third is 30. This will always be the case unless you force a change.

Ordering also matters for calculations and operations. For example, if you add two vectors together, R adds them element-wise based on their positions:

```
# make two vectors
v1 <- c(1, 2, 3)
v2 <- c(10, 20, 30)

# add the vectors together
v1 + v2
```

```
## [1] 11 22 33
```

Changing the order of elements changes the results of any operation that relies on position - like adding.

```
# reverse the order of the vector
rev_v1 <- rev(v1)
# now saved under a new object name (rev_v1)

# add rev_v1 AND v2 together again
rev_v1 + v2
```

```
## [1] 13 22 31
```

```
# how different is the result?
```

Key takeaway: Always be aware of the order of elements in a vector. Many mistakes in data analysis happen when positions are assumed incorrectly.

Exercise 3.4: Create a vector of 5 numbers representing scores of five different assessments (0-100). Please give them five unique numbers.

3.3.3 Indexing vectors

Square brackets `[]` are used to extract elements from vectors. This is how we can check the ordering.

If we want to extract the *n*th (e.g. 5th) element of a vector, specify *n* within the square brackets:

```
# make a vector
my_vector <- c(12, 16, 1, 9, 0)

# find out the 5th element in this vector
my_vector[5]
```

```
## [1] 0
```

This feature can be extended to extract multiple elements from a vector. For instance if we wanted to select the first and second elements of the vector “my.vector”, we would do so in the following way:

```
# extract elements 1 to 2
my_vector[1:2]
```

```
## [1] 12 16
```

Exercise 3.5: Use indexing to the score of your third assessment. Then reverse the vector and check which position this is in.

3.3.4 Removing values

We are also able to remove the *n*th element from a vector, we can do so using the `-` command within the square brackets.

```
# remove element 2 from the vector
my_vector[-2]
```

```
## [1] 12 1 9 0
```

```
# remove elements 1 to 2 from the vector
my_vector[-c(1:2)]
```

```
## [1] 1 9 0
```

Remember, however than you will need to re-save the vector (ideally with a different object name to avoiding overwriting).

```
# print the vector again
my_vector # does it still include the 2nd element?
```

```
## [1] 12 16 1 9 0
```

```
# remove the second element and assign it to a new object 'my_vector2'
My_vector2 <- my_vector[-2] # what about now?
```

```
# print to confirm
My_vector2
```

```
## [1] 12 1 9 0
```

3.3.5 Text vectors and factors

Vectors can be also made up of **strings** (i.e. words) instead of numbers. We simply have to enclose the characters/strings by quotation marks like in the following example:

```
# make a vector based on text and print
country <- c("Japan", "USA", "France")
country
```

```
## [1] "Japan" "USA" "France"
```

Factors are the last variable type in R, otherwise known as categorical, nominal or ordinal variables. These are variables which are groupings with no clear order (e.g. apple, banana, orange - two apples are not the same as one orange), or if ordinal then groupings with an order but you don't know the size of the space between the levels (e.g. 1st, 2nd and 3rd in a race. You know that the 1st person is quicker, but you don't know by how much).

```
# make a vector of characters
blood <- c("A", "B", "O", "AB", "A", "B", "O", "O", "AB", "B")

# print
blood
```

```
## [1] "A" "B" "O" "AB" "A" "B" "O" "O" "AB" "B"
```

```
# check class
class(blood)
```

```
## [1] "character"
```

In the first line we made a string list the blood type of 10 observations. This doesn't necessarily make a factor, however. A factor must have levels (e.g. the groups - A, B, AB, O) specified to 'lump' together these observations (e.g. 2 observations of A, 3 observations of O and 2 observations of AB)

So, we force the vector of characters to be factor using the function **as.factor**. By doing this that R recognises that blood is made up of a limited (4) number of possible values (A, AB, B and O) which become the levels. when you print **blood** now it tells you the observations in the first line, and the number of levels in the second.

```
# Froce blood to be a factor
blood <- as.factor(blood)
```

```
# print
blood
```

```
## [1] A B O AB A B O O AB B
## Levels: A AB B O
```

You can also ask R to just tell you the levels using the function `levels()` and the number of levels using `nlevels()`

```
# check levels
levels(blood)
```

```
## [1] "A" "AB" "B" "O"
```

```
## [1] "A" "AB" "B" "O"
```

```
# count number of levels
nlevels(blood)
```

```
## [1] 4
```

Factors can also be comprised of numbers. Sometimes, in data entry, rather than write ‘man’, ‘woman’ or ‘non-binary’ researchers will code these observations to ‘1’, ‘2’ and ‘3’. Some people just prefer this in data process and analysis. In this case, you just need to tell R that these numbers need to be handled as a factor.

3.3.6 Activity 3.2: Making a factor

To make a `gender` factor please do the following steps:

- Make a new object called `gender` based on a vector of 15 integers
- The observations should be a combination of 1 for men, 2 for women and 3 for non-binary. You can pick the distribution of numbers
- Check it’s class. Is it numeric or character?
- Force it to be a factor.
- Check it’s class again. What is it now?
- How many levels does it have?
- What happens to the `gender` variable now if you try to add 2 to it (e.g. `gender + 2`) - why?

Factors are very useful things, and we’ll use them a lot in this module: they’re the main way to represent a nominal scale variable so you have to make sure you are handelling them correctly in R.

3.3.7 Ordered factors

If you are working with an ordinal variable (e.g. educational level or class, in which some levels are greater than others) you will need to tell R their order because it will not necessarily know. You have to tell R in the argument of the function that it is ordered using `ordered = TRUE`

```
educ <- c("Prim", "Ter", "Sec", "Sec", "Ter", "Prim")
educ <- factor(educ, ordered=TRUE)
educ
```

```
## [1] Prim Ter  Sec  Sec  Ter  Prim
## Levels: Prim < Sec < Ter
```

In fact, with strings, R orders them automatically alphabetically. This worked for the above example between Primary education is before secondary etc. But what this means it that it will assume that A-levels (for instance) is 'before' GCSE, while we know this is not the case. In this case, you need to tell R which comes first. This requires one more argument in your code:

```
# Example vector of education levels
edu <- c("UNIVERSITY", "GCSE", "POSTGRAD", "ALEVEL", "GCSE", "UNIVERSITY", "ALEVEL")

# Convert to ordered factor with the desired order in the argument
edu_ordered <- factor(edu,
                      levels = c("GCSE", "ALEVEL", "UNIVERSITY", "POSTGRAD"),
                      ordered = TRUE)

# Check the result
edu_ordered
```

```
## [1] UNIVERSITY GCSE      POSTGRAD  ALEVEL    GCSE      UNIVERSITY ALEVEL
## Levels: GCSE < ALEVEL < UNIVERSITY < POSTGRAD
```

3.4 Data frames

So far, we have made lists of single variables (e.g. gender or education). However, when working with real data, we usually have multiple variables measured on multiple observations. In R, **this is a data frame**, which stores this information in a rectangular structure.

This is the main format of data you will be working with and we already covered how to import it last week [i.e. `mydata <- read.csv("data/mydata.csv")`].

This week we will discuss how to explore an imported data frame, how to change the name of variables, and how to sub-set a data frame.

3.4.1 Exploring a data frame

Before any data analysis, it is good to know the details about the dataset you are using, e.g. number of variables, sample size, etc.

We will use the built-in iris dataset.

```
# Look at the top of the dataset
head(iris)
```

```
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
## 1         5.1         3.5          1.4         0.2  setosa
## 2         4.9         3.0          1.4         0.2  setosa
## 3         4.7         3.2          1.3         0.2  setosa
```

```
## 4      4.6      3.1      1.5      0.2 setosa
## 5      5.0      3.6      1.4      0.2 setosa
## 6      5.4      3.9      1.7      0.4 setosa
```

```
# what are the names of the columns
names(iris)
```

```
## [1] "Sepal.Length" "Sepal.Width"  "Petal.Length" "Petal.Width"  "Species"
```

```
# how many columns are there?
ncol(iris) # e.g. variables
```

```
## [1] 5
```

```
# how many rows are there
nrow(iris) # e.g. observations
```

```
## [1] 150
```

All these functions allow you to quickly understand the size and structure of the dataset.

Question 3.1: How many observations does the iris dataset contain? How many variables?

To look at individual columns or variables you can use the \$ sign. The object before the \$ is the data frame (e.g. `iris`) and the object after is the variable name in the dataset (e.g. `Sepal.Length`)

```
# view Sepal.length in iris df
iris$Sepal.Length
```

```
## [1] 5.1 4.9 4.7 4.6 5.0 5.4 4.6 5.0 4.4 4.9 5.4 4.8 4.8 4.3 5.8 5.7 5.4 5.1
## [19] 5.7 5.1 5.4 5.1 4.6 5.1 4.8 5.0 5.0 5.2 5.2 4.7 4.8 5.4 5.2 5.5 4.9 5.0
## [37] 5.5 4.9 4.4 5.1 5.0 4.5 4.4 5.0 5.1 4.8 5.1 4.6 5.3 5.0 7.0 6.4 6.9 5.5
## [55] 6.5 5.7 6.3 4.9 6.6 5.2 5.0 5.9 6.0 6.1 5.6 6.7 5.6 5.8 6.2 5.6 5.9 6.1
## [73] 6.3 6.1 6.4 6.6 6.8 6.7 6.0 5.7 5.5 5.5 5.8 6.0 5.4 6.0 6.7 6.3 5.6 5.5
## [91] 5.5 6.1 5.8 5.0 5.6 5.7 5.7 6.2 5.1 5.7 6.3 5.8 7.1 6.3 6.5 7.6 4.9 7.3
## [109] 6.7 7.2 6.5 6.4 6.8 5.7 5.8 6.4 6.5 7.7 7.7 6.0 6.9 5.6 7.7 6.3 6.7 7.2
## [127] 6.2 6.1 6.4 7.2 7.4 7.9 6.4 6.3 6.1 7.7 6.3 6.4 6.0 6.9 6.7 6.9 5.8 6.8
## [145] 6.7 6.7 6.3 6.5 6.2 5.9
```

So we can use some of the functions we learnt earlier like `class()` in the context of the data frame on a specific variable

```
# check the class of variables in a data frame
class(iris$Sepal.Length)
```

```
## [1] "numeric"
```

```
class(iris$Species)
```

```
## [1] "factor"
```

3.4.2 Exploring with comparison operators

We can also subset by condition (e.g. greater than, equal to, less than). But before we do this **we need to know about comparison operators**.

In R, we often want to compare values in our data. R provides several comparison operators that return TRUE or FALSE depending on whether a condition is met. Here are the main ones:

Operator	Meaning	Example
<	Less than	5 < 10 returns TRUE
>	Greater than	5 > 10 returns FALSE
<=	Less than or equal to	5 <= 5 returns TRUE
>=	Greater than or equal to	5 >= 10 returns FALSE
==	Equal to	5 == 5 returns TRUE
!=	Not equal to	5 != 5 returns FALSE

In R, the command ! always indicates logical negation: i.e. the opposite (!= 5 means show me everything that isn't 5 in the dataset).

In R, you can apply these comparisons to vectors or columns in a data frame. This is like asking questions from the data, which will give you logical responses (TRUE/FALSE). For example, using the iris dataset:

```
# Which Sepal.Length values are less than 5?  
iris$Sepal.Length < 5
```

```
## [1] FALSE TRUE TRUE TRUE FALSE FALSE TRUE FALSE TRUE TRUE FALSE TRUE  
## [13] TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE  
## [25] TRUE FALSE FALSE FALSE FALSE TRUE TRUE FALSE FALSE FALSE TRUE FALSE  
## [37] FALSE TRUE TRUE FALSE FALSE TRUE TRUE FALSE FALSE TRUE FALSE TRUE  
## [49] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE  
## [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE  
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [145] FALSE FALSE FALSE FALSE FALSE FALSE
```

```
# Which Sepal.Width values are greater than or equal to 3.5?  
iris$Sepal.Width >= 3.5
```

```
## [1] TRUE FALSE FALSE FALSE TRUE TRUE FALSE FALSE FALSE FALSE TRUE FALSE  
## [13] FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE FALSE TRUE TRUE FALSE  
## [25] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE TRUE FALSE FALSE  
## [37] TRUE TRUE FALSE FALSE TRUE FALSE FALSE TRUE TRUE FALSE TRUE FALSE
```

```
## [49] TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [61] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [73] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [85] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [97] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE
```

```
# Check if Species is equal to "versicolor"
iris$Species == "versicolor"
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [25] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [49] FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [73] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [85] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [97] TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [109] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [121] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [133] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [145] FALSE FALSE FALSE FALSE FALSE FALSE
```

Each of these commands returns a logical vector (TRUE or FALSE) for each row.

3.5 Re-labelling variables

Clear and consistent variable names make your code easier to read and prevent errors. R is case-sensitive, which means `Sepal.Length` is different from `sepal.length`. A common approach is to use all lowercase letters and replace dots or spaces with underscores.

I find it simpler never to use caps in R then I never need to wonder if anything isn't working because I did, or didn't use caps.

The `dplyr` package provides a convenient `rename()` function we are going to use

```
# load the dplyr package
library(dplyr)

# Rename Sepal.Length to sepal_len
Iris2 <- iris %>%
  rename(sepal_len = Sepal.Length)

# Check the result
names(Iris2)
```

```
## [1] "sepal_len" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"
```

You can see the first variable has now been renamed. We can do this with all the variables when we hate caps.

```
# remove caps from all variables
Iris2<- iris %>%
  rename(
    sepal_length = Sepal.Length,
    sepal_width  = Sepal.Width,
    petal_length = Petal.Length,
    petal_width  = Petal.Width,
    species      = Species
  )

names(Iris2)
```

```
## [1] "sepal_length" "sepal_width" "petal_length" "petal_width" "species"
```

NB – dplyr has a nice function where it will automatically remove the caps on all variables! Let's use this to save typing

```
# Convert all column names in iris to lowercase
Iris2 <- iris %>%
  rename_with(tolower)

# Check the result
names(Iris2)
```

```
## [1] "sepal.length" "sepal.width" "petal.length" "petal.width" "species"
```

*#NB*2 - you can also do this for annoying dots with underscores. You can do this with gsub() inside ren*

```
Iris2 <- iris %>%
  rename_with(~ gsub("\\.", "_", .))

# Check the result
names(Iris2)
```

```
## [1] "Sepal_Length" "Sepal_Width" "Petal_Length" "Petal_Width" "Species"
```

3.6 Introducing pipes

In the above code you will notice we introduced the %>% character for the first time. **This is called a pipe.** The pipe operator %>%, from the dplyr package, passes the output of one step directly into the next step. Think of it as saying:

“Take this, and then do that next.” *all in one code line*

It allows us to write sequences of operations in a readable, left-to-right way.

- **Readable:** You can read it top-to-bottom, like a list of instructions.
- **No temporary variables needed:** No need to create iris_subset first.
- **Easy to extend:** You can add more steps at the end without changing the earlier code.

We will learn more about pipes next week and in future weeks as we start using them more!

4 Data Manipulation in R: Week 4

This week we move from organising data to actively manipulating it. Real datasets are rarely ready for analysis: variables need to be recoded, summaries created, formats changed and missing data handled.

4.1 Overview: Why do we need data manipulation?

In an ideal world, every dataset would arrive perfectly organised, with clear variable names, no missing values, and exactly the structure needed for analysis. In reality, this almost never happens. Data manipulation refers to all the steps we take to reshape, clean and transform raw data into a form that can actually answer our research questions. It sits between importing data and analysing data.

4.1.1 Raw data are rarely analysis-ready

Real datasets often contain problems such as:

- Variables stored in the wrong format (e.g. numbers saved as text)
- Several measurements spread across multiple columns
- Categories with confusing labels (e.g. `1 = male`, `2 = female`, `3 = other`)
- Missing values coded as `-99` or blank cells
- Five response categories where two have only one participant (small sample sizes because there are too many categories)
- Separate files for different time points or conditions

Before we can run even a simple analysis, like calculating a mean or making a plot, these issues must be resolved.

4.1.2 What counts as “manipulation”?

Typical manipulation tasks include:

- Creating new variables from existing ones
- Recoding categories into clearer labels
- Combining several items into a scale score
- Reshaping data from wide to long format
- Handling missing values correctly
- Producing summaries by group

These steps do not change the meaning of the data - they make the data usable.

4.1.3 Manipulation is part of thinking

Data manipulation is not just technical work; it is part of the research process. For example:

- Deciding to combine “very satisfied” and “satisfied” into one group reflects a theoretical judgement about what distinctions matter.
- Choosing to average three height measures assumes they represent the same underlying construct.
- Collapsing a 5-level variable into 3 levels may be necessary because small groups would make statistical results unreliable.

Good manipulation therefore requires both coding skills and research reasoning.

4.1.4 From workflow to analysis

This week we move to a more powerful approach using the **tidyverse** (an extra package in R we installed before), which allows us to express manipulation steps in a clear, readable sequence. By the end of this week you should be able to:

- Use core tidyverse ‘verbs’ (`mutate`, `group_by`, `summarise`, `select`, `filter`)
- Convert data between wide and long formats
- Recode variables and create new ones
- Set reference levels for factors
- Rename variables consistently
- Round numbers appropriately
- Handle missing values (`NA`) safely

4.2 What is tidy and clean data?

4.2.1 Tidy data

A dataset is tidy when:

- Each row = one observation
- Each column = one variable
- Each cell = one value

Example of tidy structure:

id	height	weight	gender
1	172	68	F
2	181	75	M

Untidy data mixes measurements across columns or repeats variables across rows.

4.2.2 Clean data

Even tidy data may not be clean. Clean data usually means:

- Correct variable types (numeric / factor / character)
- Informative factor labels
- Sensible categories (no tiny groups of $n=1$)
- Missing data handled explicitly

4.3 The tidyverse: transforming variables and datasets

The tidyverse is not a single package but a collection of packages designed to work together for data science in R. The core idea is that data analysis should be:

- readable like plain language
- consistent across different tasks
- based on tidy, well-structured data

- easy to combine into step-by-step workflows

Instead of learning dozens of unrelated functions with different rules, the tidyverse gives you a unified grammar for working with data frames. Most of what we do—cleaning data, creating variables, calculating summaries—can be expressed using a small set of verbs that behave in predictable ways.

We installed it last week, but make sure its loaded:

```
library(tidyverse)
```

4.3.1 What packages are inside the tidyverse?

When you run: `library(tidyverse)` R actually loads several packages:

Package	Main purpose
dplyr	Data manipulation (select, filter, mutate, summarise)
ggplot2	Data visualisation
readr	Importing data
tidyr	Reshaping data (wide - long)
tibble	Modern version of data frames
stringr	Working with text
forcats	Working with factors

For now we will be working with the dplyr, ggplot2 and tidyr packages.

4.3.2 Thinking in “verbs”

The tidyverse provides a set of functions (often called verbs) designed specifically for working with data frames. Each verb does one clear job, and verbs can be combined using pipes to create readable workflows.

The most important verbs are:

Verb	What it does	Everyday meaning
<code>select()</code>	Choose columns	“Keep these variables”
<code>filter()</code>	Choose rows	“Keep these observations”
<code>arrange()</code>	Reorder rows	“Sort the data”
<code>mutate()</code>	Create new variables	“Add or transform columns”
<code>summarise()</code>	Calculate summaries	“Reduce to statistics”
<code>group_by()</code>	Work within groups	“Do this per category”

We talked about pipes a bit last week, but going to re-cap now as fundamental in tidyverse. It means:

Take the thing on the left and use it as the first argument of the function on the right.

The pipe lets us write data analysis the same way across different functions/verbs.

Together these verbs allow you to express almost any data manipulation in plain, logical steps. Today we will be covering all but `arrange()`.

So, for today we will practice some tidyverse verbs on a simulated dataframe about student scores. Please download the dataframe from week 4 Brightspace material.

```
# import the dataset from Brightspace
df <- read_csv("data/tidyversedf.csv")
```

```
## Rows: 200 Columns: 16
## -- Column specification -----
## Delimiter: ","
## chr (1): group
## dbl (15): student_id, mark, attendance, gender_code, h1, h2, h3, lik1, gad1,...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# check it
names(df)
```

```
## [1] "student_id" "group"      "mark"      "attendance" "gender_code"
## [6] "h1"         "h2"        "h3"        "lik1"       "gad1"
## [11] "gad2"       "gad3"      "gad4"      "gad5"       "gad6"
## [16] "gad7"
```

```
nrow(df)
```

```
## [1] 200
```

```
ncol(df)
```

```
## [1] 16
```

Remember: make sure the df is saved in your data folder. Remember, to change the code if your 'data' folder is called 'Data'.

This dataset is of 200 observations of 17 variables collected from students. Each student has a unique `student_id`. The students are allocated to three groups (A = psychology, B = anthropology and C = sociology in the `group` variable). Their final mark on their stats exam is in the `mark` column. The proportion of sessions attended is in the `attendance` column. Their gender is stored under `gender_code`. Their height was measured three times to reduce measurement error and captured in variables `h1`, `h2` and `h3`. There we asked a 5-point Likert scale on how satisfied they were with the teaching on the module (variable `lik1`) and finally answered 7 questions of the GAD-7 psychometric scale to estimate their anxiety prior to the exam. The 7 questions for GAD-7 are stored under `gad1` to `gad7` variables.

We will do a number of transformations to this dataset now using tidyverse verbs

4.3.3 `filter()` – choosing rows in a dataset

One of the most common tasks in data analysis is to focus only on the observations you care about but reducing down the dataset. You might want to:

- look only at students from one tutorial group
- analyse only people who passed the exam
- remove participants with very low attendance

- focus on data from one year, one country, or one condition

`filter()` keeps rows that meet a logical condition and discards the rest. SO you can read:

```
filter(mark >= 40)
```

as: “Keep only the rows where mark is greater than or equal to 40.”

This is very different from `select()`, which chooses columns. Let’s give this a go now:

```
df %>% # pipe so = take the df (on the left of the %>%) and
  filter(mark >= 40) # do a filter by mark equal to or higher than 40
```

```
## # A tibble: 195 x 16
##   student_id group  mark attendance gender_code   h1    h2    h3  lik1  gad1
##   <dbl> <chr> <dbl>      <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1         1 1 B    80.9      94.9          1 164. 168. 171.    4    3
## 2         2 2 A    58.4      85.1          3 172. 171. 162.    3    2
## 3         3 3 A    45.2      83.9          3 171. 169. 182.    1    0
## 4         4 4 C    87.1      65.5          1 161. 146. 169.    1    1
## 5         5 5 B    80.7      76.8          2 163. 156. 172.    5    0
## 6         6 6 B    61.4      77.2          3 185. 162. 157.    1    1
## 7         7 7 C    81.6      73           1 164. 167. 160.    3    3
## 8         8 8 A    79.1      77.3          2 158 150. 177.    1    2
## 9         9 9 C    49.4      86.6          1 172. 171. 172.    4    2
## 10        10 10 A   48.2      89.5          2 175 158. 180.    5    1
## # i 185 more rows
## # i 6 more variables: gad2 <dbl>, gad3 <dbl>, gad4 <dbl>, gad5 <dbl>,
## #   gad6 <dbl>, gad7 <dbl>
```

This returns only students who achieved at least 40 which is $n = 195$. Remember, in R does not save the changes you made as a new object (or overwrite the existing one) unless you tell it to. For now, the original `df` is unchanged, R just printed the changes. You can check this by checking the number of observations in the `df`.

```
# check number of rows (observations)
nrow(df) # should be n = 195
```

```
## [1] 200
```

So we need to save it as a new object. As a rule you should be overwriting your `df`, but making a new one

```
passed_students <- df %>%
  filter(mark >= 40)
nrow(passed_students)
```

```
## [1] 195
```

You now have a new dataset containing only those 195 students who passed.

4.3.3.1 Using more than one condition Most real analyses require multiple rules at once. Now you need to use the AND (&) condition which means BOTH things must be TRUE for the observation to be selected.

```
# filter on group A AND mark more than or equal to 60
df %>%
  filter(group == "A" & mark >= 60)
```

```
## # A tibble: 51 x 16
##   student_id group  mark attendance gender_code   h1    h2    h3  lik1  gad1
##   <dbl> <chr> <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1         8 A      79.1      77.3         2  158  150.  177     1     2
## 2        12 A      89.7      89.5         1  169.  166.  174.     2     3
## 3        15 A      68.5      67.3         2  166.  170.  180.     3     3
## 4        19 A      63.6      85.4         3  161.  182.  154.     3     1
## 5        20 A      71.8      84.6         3  162.  166.  153.     4     0
## 6        24 A      61.8      93.8         3  169.  160.  172.     3     2
## 7        33 A      81.4      77.8         2  176.  163.  167.     5     3
## 8        36 A      66.9      87.4         1  179.  169.  164.     4     3
## 9        37 A      61.0      83.2         2  171.  144.  164.     4     1
## 10       43 A      67.3      91.8         2  174.  169.  158.     1     3
## # i 41 more rows
## # i 6 more variables: gad2 <dbl>, gad3 <dbl>, gad4 <dbl>, gad5 <dbl>,
## #   gad6 <dbl>, gad7 <dbl>
```

Question 4.1: what is the number of rows now in the df?

You can also use the OR (|) when either or can be TRUE

```
# filter on mark OR attendance
df %>%
  filter(mark >= 70 | attendance >= 90)
```

```
## # A tibble: 96 x 16
##   student_id group  mark attendance gender_code   h1    h2    h3  lik1  gad1
##   <dbl> <chr> <dbl>     <dbl>     <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1         1 B      80.9      94.9         1  164.  168.  171.     4     3
## 2         4 C      87.1      65.5         1  161.  146.  169.     1     1
## 3         5 B      80.7      76.8         2  163.  156.  172.     5     0
## 4         7 C      81.6      73         1  164.  167.  160.     3     3
## 5         8 A      79.1      77.3         2  158  150.  177     1     2
## 6        12 A      89.7      89.5         1  169.  166.  174.     2     3
## 7        14 C      82.2      68.1         3  163.  171.  158.     5     3
## 8        20 A      71.8      84.6         3  162.  166.  153.     4     0
## 9        22 B      91.9     105.         2  157.  160.  173.     5     0
## 10       24 A      61.8      93.8         3  169.  160.  172.     3     2
## # i 86 more rows
## # i 6 more variables: gad2 <dbl>, gad3 <dbl>, gad4 <dbl>, gad5 <dbl>,
## #   gad6 <dbl>, gad7 <dbl>
```

So this keeps students who either had a high mark (70 or above), or had high attendance, or both.

Key take-aways for filter()

- `filter()` works on rows (observations)
- you have to tell it to save a new data frame
- Conditions must evaluate to `TRUE/FALSE`
- You can combine rules with `&` and `|`

4.3.4 `select()` – choosing columns

While `filter()` chooses rows (observations), the verb `select()` chooses columns (variables). This is useful when:

- your dataset has many variables but you only need a few
- you want to simplify a table before analysis
- you want to create a cleaner dataset for sharing

You can think of `select()` = “keep these variables, drop the rest”

4.3.4.1 Keeping columns Using our student dataset if we want to just cut it down to three columns/variables (`id`, `group` and `mark`) we can type;

```
# select id, group and mark
df %>%
  select(student_id, group, mark)
```

```
## # A tibble: 200 x 3
##   student_id group  mark
##       <dbl> <chr> <dbl>
## 1         1 B     80.9
## 2         2 A     58.4
## 3         3 A     45.2
## 4         4 C     87.1
## 5         5 B     80.7
## 6         6 B     61.4
## 7         7 C     81.6
## 8         8 A     79.1
## 9         9 C     49.4
## 10        10 A     48.2
## # i 190 more rows
```

```
# This returns a new data frame containing only those three columns.
# To save it as a new df:
```

```
df_small <- df %>%
  select(student_id, group, mark)

# check the top 6 lines of the new smaller df
head(df_small)
```

```
## # A tibble: 6 x 3
##   student_id group  mark
##       <dbl> <chr> <dbl>
## 1         1 B     80.9
```

```
## 2      2 A      58.4
## 3      3 A      45.2
## 4      4 C      87.1
## 5      5 B      80.7
## 6      6 B      61.4
```

4.3.4.2 Removing columns Removing columns is just as easy with `select()` - you just use the minus sign:

```
# to keeps everything except attendance.
df2 <- df %>%
  select(-attendance)

# view the new df
head(df2)
```

```
## # A tibble: 6 x 15
##   student_id group mark gender_code h1 h2 h3 lik1 gad1 gad2 gad3
##   <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      1 B      80.9      1 164. 168. 171. 4 3 0 3
## 2      2 A      58.4      3 172. 171. 162. 3 2 2 3
## 3      3 A      45.2      3 171. 169. 182. 1 0 2 0
## 4      4 C      87.1      1 161. 146. 169. 1 1 0 0
## 5      5 B      80.7      2 163. 156. 172. 5 0 2 3
## 6      6 B      61.4      3 185. 162. 157. 1 1 2 0
## # i 4 more variables: gad4 <dbl>, gad5 <dbl>, gad6 <dbl>, gad7 <dbl>
```

```
# You can drop several variable just add to the list using
# the vector function you learnt last week (c())

# drop attendance and mark
df3 <- df %>%
  select(-c(attendance, mark))

# view the new df
head(df3)
```

```
## # A tibble: 6 x 14
##   student_id group gender_code h1 h2 h3 lik1 gad1 gad2 gad3 gad4
##   <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      1 B      1 164. 168. 171. 4 3 0 3 2
## 2      2 A      3 172. 171. 162. 3 2 2 3 3
## 3      3 A      3 171. 169. 182. 1 0 2 0 2
## 4      4 C      1 161. 146. 169. 1 1 0 0 0
## 5      5 B      2 163. 156. 172. 5 0 2 3 2
## 6      6 B      3 185. 162. 157. 1 1 2 0 3
## # i 3 more variables: gad5 <dbl>, gad6 <dbl>, gad7 <dbl>
```

Whether you select columns to keep or remove just depends the number you want to remove from the original data frame. If you want to keep lots then use 'remove', only a few, then use 'keep' - just saves on your typing.

4.3.5 mutate() – creating and recoding variables

In psychology we almost never analyse raw variables exactly as they were entered. We usually need to:

- add items together to create scale scores
- convert raw measurements into meaningful indicators
- recode numeric codes into labels people can understand
- collapse small groups to avoid unstable statistics

The tidyverse verb `mutate()` is the main tool for doing all of this - it creates new columns or change existing ones.

You can think of `mutate()` = “add a new variable based on what is already there”

Rather than editing the raw data in Excel, we create new variables inside R so the process is reproducible. We will now go through some key examples you will need to use in psychology.

4.3.5.1 mutate() to % or logical TRUE/FALSE Psychologists often need to create relative scores (%) from absolute numbers as well as create indicator variables:

- clinical cut-offs
- high vs low groups
- passed vs failed

Let’s practice this now with our student dataset.

```
# in the original df make two new variables
df %>%
  mutate(
    mark_percent = mark / 100, # mark as a %
    passed = mark >= 40 # TRUE/FALSE if passed
  )
```

```
## # A tibble: 200 x 18
##   student_id group  mark attendance gender_code   h1   h2   h3  lik1  gad1
##   <dbl> <chr> <dbl>   <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1         1 B     80.9    94.9         1 164. 168. 171.    4    3
## 2         2 A     58.4    85.1         3 172. 171. 162.    3    2
## 3         3 A     45.2    83.9         3 171. 169. 182.    1    0
## 4         4 C     87.1    65.5         1 161. 146. 169.    1    1
## 5         5 B     80.7    76.8         2 163. 156. 172.    5    0
## 6         6 B     61.4    77.2         3 185. 162. 157.    1    1
## 7         7 C     81.6     73         1 164. 167. 160.    3    3
## 8         8 A     79.1    77.3         2 158 150. 177    1    2
## 9         9 C     49.4    86.6         1 172. 171. 172.    4    2
## 10        10 A     48.2    89.5         2 175 158. 180    5    1
## # i 190 more rows
## # i 8 more variables: gad2 <dbl>, gad3 <dbl>, gad4 <dbl>, gad5 <dbl>,
## #   gad6 <dbl>, gad7 <dbl>, mark_percent <dbl>, passed <lgl>
```

```
# has it saved?
head(df)
```

```
## # A tibble: 6 x 16
##   student_id group  mark attendance gender_code   h1   h2   h3  lik1  gad1
##   <dbl> <chr> <dbl>   <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1         1 B     80.9     94.9         1 164. 168. 171.    4    3
## 2         2 A     58.4     85.1         3 172. 171. 162.    3    2
## 3         3 A     45.2     83.9         3 171. 169. 182.    1    0
## 4         4 C     87.1     65.5         1 161. 146. 169.    1    1
## 5         5 B     80.7     76.8         2 163. 156. 172.    5    0
## 6         6 B     61.4     77.2         3 185. 162. 157.    1    1
## # i 6 more variables: gad2 <dbl>, gad3 <dbl>, gad4 <dbl>, gad5 <dbl>,
## #   gad6 <dbl>, gad7 <dbl>
```

This creates two new columns:

- mark_percent – the original mark divided by 100-
- passed – TRUE/FALSE depending on the mark being above 40 (pass) or not (Fail).

Question 4.2 - what do you need to do to save these new variables in the df? Do this now.

Question 4.3 - How do you check the class of the two new variables you made? What are they?

4.3.5.2 Creating Psychometric Scale Scores Most questionnaires measure constructs (e.g., anxiety, depression and well-being) using multiple items, like the GAD-7. We normally need to combine (sum) these into a single score. We can also do this with `mutate()`

This will be highly useful because psychological constructs are rarely measured by one question

```
# compute total GAD7 score from several items
df <- df %>%
  mutate(total = gad1 + gad2 + gad3 + gad4 + gad5 + gad6 + gad7)

# note as we're just adding new variables we don't need to give the df a new name

# check df variable names
names(df)
```

```
## [1] "student_id" "group"      "mark"      "attendance" "gender_code"
## [6] "h1"         "h2"         "h3"         "lik1"       "gad1"
## [11] "gad2"       "gad3"       "gad4"       "gad5"       "gad6"
## [16] "gad7"       "total"
```

```
# NB - if you want to type less you can tell row to sum between gad1-gad7
df <- df %>%
  mutate(total = rowSums(across(gad1:gad7)))

# check df variable names
names(df)
```

```
## [1] "student_id" "group"      "mark"      "attendance" "gender_code"
## [6] "h1"         "h2"         "h3"         "lik1"       "gad1"
## [11] "gad2"       "gad3"       "gad4"       "gad5"       "gad6"
## [16] "gad7"       "total"
```

Question 4.4 - is `total` the best name for this new variable? Why not?

What this code does:

- Adds a new variable called `total`
- For each participant it adds their responses to items `gad1-7`
- The original item variables stay unchanged

Tip: it's always useful to reduce typing where you can because the more you type the more errors can sneak in. `gad1:gad7` is better than `gad1, gad2, gad3...`

4.3.5.3 Mean of repeated measures Sometimes we have several measurements of the same thing (e.g., height measured 3 times, reaction time across trials). It is unlikely you will want to run three statistical models (and even if you did, how would you reconcile different results?) so we will want to reduce these 3 measures into a single observation.

One way of doing this is computing the average of the three scores, which we can also do with `mutate`

```
# compute mean of h1, h2 and h3
df <- df %>%
  mutate(mean_height = (h1 + h2 + h3) / 3) # new variable is called mean_height

# check the df
names(df)
```

```
## [1] "student_id" "group"      "mark"      "attendance" "gender_code"
## [6] "h1"         "h2"         "h3"         "lik1"       "gad1"
## [11] "gad2"       "gad3"       "gad4"       "gad5"       "gad6"
## [16] "gad7"       "total"      "mean_height"
```

NB – obviously like we learnt in previous weeks we will fit an issue if there are NAs in the dataset and in this code we haven't told R how to handle these NAs so it will come up with an error. So we can use this slightly fancier instead

```
# compute mean of height removing NAs
df <- df %>%
  mutate(mean_height = rowMeans(across(h1:h3), na.rm = TRUE))

# check the df
names(df)
```

```
## [1] "student_id" "group"      "mark"      "attendance" "gender_code"
## [6] "h1"         "h2"         "h3"         "lik1"       "gad1"
## [11] "gad2"       "gad3"       "gad4"       "gad5"       "gad6"
## [16] "gad7"       "total"      "mean_height"
```

This one handles the NA (tells R to remove them) and also computes the mean across `h1` to `h3` without typing all of them out (which is important if you're computing the mean across a bunch of variables).

4.3.5.4 Recoding numbers into meaningful labels Datasets often contain numeric codes rather than words

- 1 = man, 2 = women
- 0 = control, 1 = intervention1, 2 = intervention2
- 1-5 Likert scales rather than “strongly agree” etc

Numbers are convenient for computers but terrible for humans reading output. It can make it much harder to interpret your results with this, and you may forget your coding scheme later on, making it hard to reproduce your work. So we can use `mutate` to convert numeric codes into words.

```
# make a new variable gender based on the old gender_code
df <- df %>%
  mutate(
    gender = case_when(
      gender_code == 1 ~ "man", # if gender_Code 1 then make it man
      gender_code == 2 ~ "woman", # if gender_Code 2 then make it woman
      gender_code == 3 ~ "non-binary", # if gender_Code 3 then make it non-binary
      TRUE ~ NA_character_
    )
  )
```

To break this longer code down:

- `case_when()`, each line is: `condition ~ value_if_condition_is_true` then make it either man, woman or non-binary.
- The final line: `TRUE ~ NA`: “For all remaining cases that didn’t match any rule above → assign NA

4.3.5.5 Collapsing Small Categories Sometimes, once you have collected your data you see that in a factor, some levels have very small sample sizes. For instance, imagine this variable:

Education	n
GCSE	4
A-level	5
UG	82
PG	91

Running statistics with tiny groups (n=4, n=5) is unstable, violates assumptions and is hard to interpret. The solution to this is to collapse categories (e.g. GCSE and A-level can be combined to a single level)

We can do this in the practice df by combining levels in the Likert scale. Likert scales are often scored 1-5, representing 1 strong disagree, 2 = disagree, 3 neither agree or disagree, 4 = agree and 5 = strongly agree.

But five levels is a lot to handle, and you may have low responses in 1 (strongly disagree) and 5 (strongly agree) so what we want to do is:

- 1) Combine 1 and 2 to = **disagree**
- 2) Combine 4 and 5 to = **agree**
- 3) Keep 3 as = **neither**

So 5 levels has become 3, which is much easier to handle and can improve statistical power:

```

# make a new variable
df <- df %>%
  mutate(
    likred = case_when(
      lik1 %in% c(1, 2) ~ "disagree", # saying 1 & 2 is disagree
      lik1 %in% c(4, 5) ~ "agree", # saying 4 & 5 is agree
      lik1 == "3" ~ "neither" # 3 = neither
    )
  )

# check new variable levels
levels(df$likred)

```

```
## NULL
```

```

# recode as a factor
df$likred <- as.factor(df$likred)

# re check levels
levels(df$likred)

```

```
## [1] "agree" "disagree" "neither"
```

So what happened here was that R did not automatically make this new variable into a factor with levels. So we had to force it to change the type which we learnt in week 3.

4.3.5.6 Rounding numbers One last important transformation you might want to do to your data is round the numbers to a specific number of decimal points.

This is useful because lots of decimal points can be hard to digest and also gives a false sense of precision. We often round numbers to 2 decimal points and p-values to 3.

With mutate we can just do this with:

```

# look at mark
df$mark

```

```

## [1] 80.93775 58.35973 45.19076 87.14568 80.74939 61.42386 81.58477 79.12445
## [9] 49.38460 48.15086 68.17019 89.65814 59.54994 82.16958 68.50772 36.43110
## [17] 38.27665 53.38741 63.62901 71.75569 50.64609 91.85747 49.67473 61.82952
## [25] 55.33423 50.03762 72.22891 60.19034 56.99837 55.82131 54.21092 62.31661
## [33] 81.44100 62.86741 36.11005 66.86383 61.04872 79.02359 73.58570 66.51097
## [41] 53.62039 83.29179 67.32400 63.27805 70.28498 41.76448 73.18074 59.94671
## [49] 65.09475 61.39583 47.01519 38.14733 67.07402 66.02878 69.35691 68.87189
## [57] 78.25075 55.31510 96.16425 65.90653 79.12850 55.13866 96.35615 55.97945
## [65] 47.12002 60.87741 68.80562 67.40285 78.98596 61.94374 62.75328 76.62810
## [73] 48.62585 62.95967 71.89341 74.42860 66.23534 53.29945 89.01658 72.86939
## [81] 87.94858 77.02210 56.03089 88.90059 74.71830 78.87313 68.87642 64.89555
## [89] 80.93132 54.18157 52.50497 68.77632 76.24425 76.57646 84.16189 53.99158
## [97] 78.98715 76.25668 60.25151 51.00435 53.66425 64.49694 77.72041 71.08499
## [105] 59.55685 58.73258 62.45110 55.38933 57.11402 61.89156 97.88393 68.76537
## [113] 76.51394 78.62201 65.73108 49.35591 55.45279 84.48683 55.23324 52.78231

```

```
## [121] 65.31940 47.88896 47.21733 66.88302 52.68736 57.00226 57.70427 58.69392
## [129] 76.28647 35.70810 54.03901 86.65348 63.43606 76.45582 97.91364 68.12605
## [137] 74.95630 72.64560 61.59824 49.67941 93.01588 74.79042 73.56741 87.43689
## [145] 57.06329 70.76893 54.98897 76.06791 64.08968 57.78978 87.85088 69.85951
## [153] 82.46269 53.31963 51.22213 68.72877 73.84093 60.80207 77.75107 49.51299
## [161] 61.60998 58.83737 88.45913 61.54572 66.71588 61.60807 67.57469 83.86103
## [169] 55.94367 72.42693 50.75751 51.79895 63.23941 61.68484 66.25382 62.25602
## [177] 65.29913 70.99419 67.85476 74.99646 62.93165 49.73841 68.84654 67.68551
## [185] 62.60950 67.59880 62.05935 68.62086 75.15851 80.29301 61.47630 70.98413
## [193] 69.20717 52.03152 74.87389 61.73618 72.95536 58.14615 61.98594 61.72284
```

```
# re-save mark with 2 decimal points
df <- df %>%
  mutate(mark = round(mark, 2)) # round is the function, mark the variable
# and 2 the number of decimal points

# check the variable
df$mark
```

```
## [1] 80.94 58.36 45.19 87.15 80.75 61.42 81.58 79.12 49.38 48.15 68.17 89.66
## [13] 59.55 82.17 68.51 36.43 38.28 53.39 63.63 71.76 50.65 91.86 49.67 61.83
## [25] 55.33 50.04 72.23 60.19 57.00 55.82 54.21 62.32 81.44 62.87 36.11 66.86
## [37] 61.05 79.02 73.59 66.51 53.62 83.29 67.32 63.28 70.28 41.76 73.18 59.95
## [49] 65.09 61.40 47.02 38.15 67.07 66.03 69.36 68.87 78.25 55.32 96.16 65.91
## [61] 79.13 55.14 96.36 55.98 47.12 60.88 68.81 67.40 78.99 61.94 62.75 76.63
## [73] 48.63 62.96 71.89 74.43 66.24 53.30 89.02 72.87 87.95 77.02 56.03 88.90
## [85] 74.72 78.87 68.88 64.90 80.93 54.18 52.50 68.78 76.24 76.58 84.16 53.99
## [97] 78.99 76.26 60.25 51.00 53.66 64.50 77.72 71.08 59.56 58.73 62.45 55.39
## [109] 57.11 61.89 97.88 68.77 76.51 78.62 65.73 49.36 55.45 84.49 55.23 52.78
## [121] 65.32 47.89 47.22 66.88 52.69 57.00 57.70 58.69 76.29 35.71 54.04 86.65
## [133] 63.44 76.46 97.91 68.13 74.96 72.65 61.60 49.68 93.02 74.79 73.57 87.44
## [145] 57.06 70.77 54.99 76.07 64.09 57.79 87.85 69.86 82.46 53.32 51.22 68.73
## [157] 73.84 60.80 77.75 49.51 61.61 58.84 88.46 61.55 66.72 61.61 67.57 83.86
## [169] 55.94 72.43 50.76 51.80 63.24 61.68 66.25 62.26 65.30 70.99 67.85 75.00
## [181] 62.93 49.74 68.85 67.69 62.61 67.60 62.06 68.62 75.16 80.29 61.48 70.98
## [193] 69.21 52.03 74.87 61.74 72.96 58.15 61.99 61.72
```

So to summarise the key ideas about mutate:

- `mutate()` always keeps the same number of rows
- it is used for creating or transforming variables
- calculations are done row-by-row
- the original data remain untouched unless you save the result

4.3.6 summarise() – summarising data

While mutate adds information to every row, `summarise()` reduces many rows into a single value.

So we can understand `summarise()` = “turn data into summary statistics”

This is how we calculate:

- means
- totals
- counts
- minimums / maximums
- standard deviations

```
#Basic example of key summaries
df %>%
  summarise( # opening bracket
    mean_mark = mean(mark), # 1st line
    max_mark = max(mark), # 2nd line
    n_students = n() # last line
  ) # closing bracket
```

```
## # A tibble: 1 x 3
##   mean_mark max_mark n_students
##   <dbl>    <dbl>    <int>
## 1     66.0     97.9        200
```

The result is one row containing the summary statistics. This has a bunch of real-life uses:

- reporting average test scores
- counting participants per condition
- describing datasets for methods sections
- checking data quality (missing values, ranges)

As above, this hasn't saved this information. So you would have to keep re-running it. If you wanted to save it and - highly recommended - save this table to your computer files to use in your report/dissertation then you will need to:

```
# calling the new object 'meansum' or the summary of mean
meansum <- df %>%
  summarise(
    mean_att = mean(attendance), # mean of mark
    max_att = max(attendance), # max of mark
    min_att = min(attendance), # min of mark
    sd_att = sd(attendance) # max of mark
  )

# Note - to add more lines I just added a , and then started a new line.
# the last line has no comma, and then has the closing bracket

# view the saved table
print(meansum)
```

```
## # A tibble: 1 x 4
##   mean_att max_att min_att sd_att
##   <dbl>    <dbl>    <dbl>    <dbl>
## 1     82.6     108.     57.5     9.30
```

```
# save it to your outputs folder

write.csv(meansum, "outputs/meansum.csv")

# again if you have **Out**puts remember to change the code to capitalise the folder name.
```

So now you have summarised one of your variables, saved this as a new object which you can save to your computer as a table for use in communicating your results!

Common summary functions include:

- `mean(mark)` # average
- `median(mark)` # middle value when ordered smallest to largest
- `sd(mark)` # standard deviation
- `min(mark)` # lowest
- `max(mark)` # highest
- `n()` # number of rows

Question 4.4: compute all of these summary statistics on the transformed variable you made earlier `mean_height` and save to your outputs folder.

4.3.7 `group_by()` - doing things within groups

On its own, `summarise()` gives one answer for the whole dataset. Usually we want answers per group (e.g. psychology versus anthropology versus sociology students). This is what `group_by()` does.

so `group_by()` is = “do the next step separately for each category”

So we can re-compute the mean, but now by student group (A, B and C) which is another variable in our dataset:

```
# make a new table for the average and sample size for each group

meanbygroup <- df %>%
  group_by(group) %>%
  summarise(
    mean_mark = mean(mark),
    median_mark = median(mark),
    n = n()
  )

# view the saved table
print(meanbygroup)
```

```
## # A tibble: 3 x 4
##   group mean_mark median_mark    n
##   <chr>    <dbl>      <dbl> <int>
## 1 A      65.6        65.1    71
## 2 B      67.3        67.6    63
## 3 C      65.0        62.2    66
```

```
# save it to your outputs folder

write.csv(meanbygroup, "outputs/meanbygroup.csv")
```

Now we get one row per group (A, B & C) instead of one row overall. So: - `group_by(group)` splits the data into invisible mini-datasets - `summarise()` runs inside each mini-dataset just like before - the results are combined back together

Question 4.5: which group has the highest mean mark? Which group has the largest sample size?

You can also group by two (or more) groups if you want:

```
# compute mean, SD and n for group&gender
grogendersum <- df %>%
  group_by(group, gender) %>%
  summarise(
    mean_mark = mean(mark),
    sd_mark = sd(mark),
    n = n()
  )
```

'summarise()' has grouped output by 'group'. You can override using the
'.groups' argument.

```
# view the saved table
print(grogendersum)
```

```
## # A tibble: 9 x 5
## # Groups:   group [3]
##   group gender    mean_mark sd_mark    n
##   <chr> <chr>         <dbl>   <dbl> <int>
## 1 A     man           65.1    14.0    19
## 2 A     non-binary     67.5    12.4    25
## 3 A     woman          64.3    10.7    27
## 4 B     man           67.3    13.8    27
## 5 B     non-binary     67.2    10.5    21
## 6 B     woman          67.7    13.2    15
## 7 C     man           65.1    12.6    25
## 8 C     non-binary     60.2     9.28    22
## 9 C     woman          70.4    16.4    19
```

```
# save it to your outputs folder

write.csv(grogendersum, "outputs/grogendersum.csv")
```

This calculates means for every possible combination of group and gender (e.g. a man in psychology, or a non-binary individual in sociology)!

Common mistakes with summarise

- Expecting `summarise` to keep all rows → it always collapses the data.
- Forgetting `group_by` before `summarise` → you get one overall mean instead of per group.
- Using `mutate` when you really want `summarise` → `mutate` repeats values instead of reducing them

4.3.8 Activity 4.1: using tidyverse verbs

This activity will bring together a bunch of the verbs you just learnt about still using the `df` dataset. Specifically we're going to explore anxiety (the GAD-7 sum, labelled `total`) by degree (variable = `group`) and exam grade (variable = `mark`). Please follow these steps to answer the following questions:

- **Step 1:** using `select()` select only the relevant variables (`total`, `group` & `mark`). Call the new dataframe `anxietydf`
- **Step 2:** relabel the variables as follows using `rename` (we did this last week in section 3.5)
 - Change name of `group` -> `degree`
 - Relabel the levels A, B and C of `degree` to psychology, anthropology and sociology, respectively, using `mutate()`
 - Change name of `total` -> `GAD7`
- **Step 3:** Using `mutate()` Convert GAD7 into a factor called `anxiety` with two levels based on scores < 10 -> "FALSE" and >= 10 -> "TRUE"
- **Step 4:** Using `group_by()` and `summarise()` compute the mean exam mark (`mark`) by BOTH variables `anxiety` and `degree`

Question 4.6: Which combination of degree and anxiety has the lowest score in the dataset?

4.4 Wide and long data

Real research data rarely arrive in the "perfect" format for analysis. The same information can be organised in at least two fundamentally different ways:

- 1) **Wide format** – one row per participant, repeated measures stored in multiple columns
- 2) **Long format** – one row per observation, with a variable that tells you which condition or time point it belongs to.

Neither format is "right" or "wrong". They simply serve different purposes. You will need to work out what format you need your data in depending on your purpose!

4.4.1 Wide data

In *wide data* each person appears in ONLY one row, and each observation or data point collected on them has a separate column. For instance, if you were measuring anxiety at three separate time points in the same participant it would look like this:

id	anxiety_t1	anxiety_t2	anxiety_t3
1	12	10	8
2	15	14	13

It's called wide data because as you collect more and more data on a participant you can imagine the dataset getting wider and wider as more columns are filled up!

This format is:

- easy for us to read as people
- similar to how online surveys (Like Qualtrics) kick out data when you have asked participants a bunch of questions
- convenient for computing summary statistics (e.g. mean anxiety score)

4.4.2 Long data

In long data, each measurement will get its own row, so participants will be spread across many rows - making your data very long:

id	time	anxiety
1	t1	12
1	t2	10
1	t3	8
2	t1	15
2	t2	14
2	t3	13

This structure is harder for us to read but is often expected in repeated measures models. For within-subject designs, where the same participant is measured more than once (paired t-test, repeated-measures ANOVA and linear mixed models) the analysis needs to know 1) which row belongs to the same person and 2) which condition/time each score represents. This only is clear in the long format.

What this means is that when you have wide data, you might need to **pivot** it to a long format.

4.4.3 Converting wide to long with `pivot_longer()`

Imagine we collected three height measurements per student as they grow over time:

```
# make a wide dataframe:

df_wide <- data.frame(
  id = 1:3, # 1 participants
  h1 = c(154, 159, 167), # first height for each participant
  h2 = c(165, 162, 179), # second height for each participant
  h3 = c(171, 163, 181) # third height for each participant
)

# view the dataframe
df_wide
```

```
##   id  h1  h2  h3
## 1   1 154 165 171
## 2   2 159 162 163
## 3   3 167 179 181
```

So to make it long we need to pivot the table so you have one column which is `id`, one column as `height` and a new column which is `time` which is when each of these height measures are taken.

```
# pivot df longer

df_long <- df_wide %>% # renames df based on the df_wide
  pivot_longer(
    cols = c(h1, h2, h3), # these are the columns to reduce to one
    names_to = "time", # this takes the names of the original columns into
    # the new variable `time`
```

```

    values_to = "height" # the values of h1-h3 are in the new variable 'height'
  )

# view the new dataset
df_long

```

```

## # A tibble: 9 x 3
##       id time  height
##   <int> <chr> <dbl>
## 1     1  h1    154
## 2     1  h2    165
## 3     1  h3    171
## 4     2  h1    159
## 5     2  h2    162
## 6     2  h3    163
## 7     3  h1    167
## 8     3  h2    179
## 9     3  h3    181

```

Now we have:

- one column saying which height measure
- one column containing the actual values

Question 4.6: how many rows did the `df_wide` have? How many rows does the `df_long` have?

4.4.4 Pivoting long to wide with `pivot_wider()`

Sometimes we need to do the opposite and change it from long format to wide. We can change the new `df_long` back to `df_wide` if we like:

```

# pivot wider
df_wide2 <- df_long %>% # rename the df df_wide2 not to overwrite
  pivot_wider( # function
    names_from = time, # the variables labelled from 'time' column
    values_from = height # values from the height column
  )

# view df to confirm
df_wide2

```

```

## # A tibble: 3 x 4
##       id   h1   h2   h3
##   <int> <dbl> <dbl> <dbl>
## 1     1  154  165  171
## 2     2  159  162  163
## 3     3  167  179  181

```

Now it is back to where it was originally. Something like this would be useful if your, like we did above, computing a total GAD-7 score from `gad1-gad7`.

4.4.5 Activity 4.2: pivoting the tidyverse df

Going back to the dataframe we were using for the tidyverse verbs, you will remember we had three variables measuring height. To pivot this from wide to long, follow these steps:

- 1) **Step 1:** use `pivot_wider()` on `df` and rename to `dflong`
- 2) **Step 2:** Name columns “measurement” in `names_to`
- 3) **Step 3:** Pull the values of height into the `heights` column with `values_to`
- 4) **Step 4:** View the new dataset and answer the following questions

Question 4.7: how many rows does this new long dataset have?

Question 4.8: for the other variables (e.g. total, gender) how are they now recorded for each participant?

- 5) **Step 5:** convert this back to from long to wide again and check it matches the original structure.

Key take-aways:

- Wide = one row per person
- Long = one row per observation
- Repeated measures almost always require long
- `pivot_longer()` and `pivot_wider()` let you move flexibly between them
- Analyses might require you to switch format depending on task

4.5 Missing data (NAs)

In real research, datasets are almost never complete. Participants skip questions, equipment fails, files get corrupted or someone withdraws from a study. These gaps are called missing data, and in R they are stored as the special value: `NA`

Missing data matter because:

- many statistical functions will not run if `NAs` are present
- means and correlations can be biased if `NAs` are recorded as 0
- sample sizes may differ across variables
- different handling choices can change conclusions

So part of data cleaning is deciding how to treat missing values in a transparent and principled way.

4.5.1 Where do missing data come from?

In psychology, common reasons include:

- Participant behaviour
 - skipped sensitive questions
 - survey fatigue near the end
 - “prefer not to say”
- Study design

- participants only complete one condition
- branching logic in surveys
- Technical issues
 - recording equipment fails
 - online form not saved
 - data entry mistakes

Understanding why data are missing helps decide what to do with them.

4.5.2 Missing values in R

R uses the special symbol `NA` (Not Available). It is not the same as:

- 0
- an empty cell
- the text "NA"

Let's practice this:

```
#calculation with NA return NA while 0 act normally:
```

```
# 5 add NA
5 + NA
```

```
## [1] NA
```

```
# 5 add 0
5 + 0
```

```
## [1] 5
```

```
# R won't calculate a mean with an NA as we know
mean(c(1, 2, NA, 4))
```

```
## [1] NA
```

```
# but it will with a 0
mean(c(1, 2, 0, 4))
```

```
## [1] 1.75
```

This is R protecting you from accidentally ignoring missingness. If it didn't tell you 'NA' you wouldn't know some data was missing.

So the first thing we have to do is check for missing data

4.5.3 Finding missing data in a dataframe

There are a few ways of checking for missingness in your dataset. For this activity we going to swap to a new dataset with NAs in it. Please download the NAdf data frame from Brightspace week 4 and save in your data folder (again remembering about capitalisation of Data versus data)

1. Check the whole dataset

```
# read new df
NAdf <- read_csv("data/NAdf.csv")

## Rows: 20 Columns: 14
## -- Column specification -----
## Delimiter: ","
## db1 (14): id, gender, mark, attendance, h1, h2, h3, gad1, gad2, gad3, gad4, ...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
# ask R 'is there any NAs' in the df object?
is.na(NAdf)
```

```
##           id gender mark attendance   h1   h2   h3 gad1 gad2 gad3 gad4
## [1,] FALSE  FALSE FALSE      FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [2,] FALSE  FALSE FALSE      FALSE FALSE FALSE TRUE  FALSE FALSE FALSE
## [3,] FALSE  FALSE TRUE      FALSE TRUE  FALSE FALSE TRUE  FALSE FALSE
## [4,] FALSE   TRUE FALSE      TRUE  FALSE FALSE FALSE FALSE TRUE  FALSE
## [5,] FALSE  FALSE FALSE      FALSE FALSE TRUE  FALSE FALSE TRUE  FALSE
## [6,] FALSE  FALSE FALSE      FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [7,] FALSE  FALSE FALSE      FALSE FALSE FALSE TRUE  FALSE FALSE FALSE
## [8,] FALSE  FALSE TRUE      FALSE TRUE  FALSE FALSE TRUE  FALSE FALSE
## [9,] FALSE  TRUE  FALSE      TRUE  FALSE FALSE FALSE FALSE FALSE FALSE
## [10,] FALSE FALSE FALSE      FALSE FALSE FALSE FALSE FALSE TRUE  FALSE
## [11,] FALSE FALSE FALSE      FALSE FALSE TRUE  FALSE FALSE FALSE TRUE
## [12,] FALSE FALSE TRUE      FALSE FALSE FALSE FALSE FALSE FALSE TRUE
## [13,] FALSE FALSE FALSE      FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [14,] FALSE FALSE FALSE      FALSE TRUE  FALSE FALSE FALSE FALSE FALSE
## [15,] FALSE  TRUE  FALSE      FALSE FALSE FALSE FALSE TRUE  FALSE FALSE
## [16,] FALSE  FALSE FALSE      TRUE  FALSE FALSE TRUE  FALSE FALSE FALSE
## [17,] FALSE  FALSE TRUE      FALSE FALSE FALSE FALSE FALSE TRUE  FALSE
## [18,] FALSE  FALSE FALSE      FALSE FALSE FALSE FALSE FALSE TRUE  FALSE
## [19,] FALSE  FALSE FALSE      FALSE FALSE TRUE  FALSE FALSE FALSE TRUE
## [20,] FALSE  FALSE FALSE      FALSE FALSE FALSE FALSE FALSE FALSE FALSE
##           gad5 gad6 gad7
## [1,] FALSE FALSE FALSE
## [2,] FALSE FALSE FALSE
## [3,] FALSE FALSE FALSE
## [4,] FALSE FALSE FALSE
## [5,] FALSE FALSE FALSE
## [6,] FALSE FALSE FALSE
## [7,] TRUE  FALSE FALSE
## [8,] FALSE TRUE  FALSE
## [9,] FALSE FALSE TRUE
```

```
## [10,] FALSE FALSE FALSE
## [11,] FALSE FALSE FALSE
## [12,] FALSE FALSE FALSE
## [13,] FALSE FALSE FALSE
## [14,]  TRUE FALSE FALSE
## [15,] FALSE  TRUE FALSE
## [16,] FALSE FALSE  TRUE
## [17,] FALSE FALSE FALSE
## [18,] FALSE FALSE FALSE
## [19,] FALSE FALSE FALSE
## [20,] FALSE FALSE FALSE
```

This returns TRUE/FALSE for every cell. As our data frame is pretty large this is hard to digest! So let's look by column.

2. Count missing per variable

```
# Asks R to sum the number of NAs in each column in the dataset
```

```
colSums(is.na(NAdf))
```

```
##      id      gender      mark attendance      h1      h2      h3
##      0         3         4         3         3         3         3
##   gad1     gad2     gad3     gad4     gad5     gad6     gad7
##      3         3         3         3         2         2         2
```

This is more useful, we can see there are no missing values in ID, but between 3-4 in other variables. But of course we don't know how many relative to the total these represent, so we can also ask R to give us the proportion of missingness for each variable:

3. Proportion missing

```
# Compute the proportion of missing values for each column in the dataset
```

```
colMeans(is.na(NAdf))
```

```
##      id      gender      mark attendance      h1      h2      h3
##   0.00     0.15     0.20     0.15     0.15     0.15     0.15
##   gad1     gad2     gad3     gad4     gad5     gad6     gad7
##   0.15     0.15     0.15     0.15     0.10     0.10     0.10
```

So we can see that missingness is highest in the `mark` column with 20%, and lowest in `gad6` and `gad7` with 10%.

Finally, we can ask R to compute summary statistics for us on the entire dataset (this is a good one to know when you first explore your data as brings lots of functions together)

4. Quick overview

```
# produce summary stats for the entire df
summary(NAdf)
```

```
##      id      gender      mark      attendance
## Min.   : 1.00   Min.   :1.000   Min.   :55.00   Min.   :78.00
## 1st Qu.: 5.75   1st Qu.:1.000   1st Qu.:64.25   1st Qu.:81.00
## Median :10.50   Median :2.000   Median :69.50   Median :84.00
## Mean   :10.50   Mean   :1.765   Mean   :70.00   Mean   :83.65
## 3rd Qu.:15.25   3rd Qu.:2.000   3rd Qu.:74.75   3rd Qu.:86.00
## Max.   :20.00   Max.   :3.000   Max.   :90.00   Max.   :90.00
##                NA's   :3      NA's   :4      NA's   :3
##      h1      h2      h3      gad1
## Min.   :165.0   Min.   :166.0   Min.   :165.0   Min.   :1.000
## 1st Qu.:168.0   1st Qu.:167.0   1st Qu.:167.0   1st Qu.:2.000
## Median :169.0   Median :168.0   Median :168.0   Median :2.000
## Mean   :168.8   Mean   :168.3   Mean   :168.2   Mean   :2.235
## 3rd Qu.:170.0   3rd Qu.:170.0   3rd Qu.:169.0   3rd Qu.:3.000
## Max.   :172.0   Max.   :171.0   Max.   :171.0   Max.   :4.000
## NA's   :3      NA's   :3      NA's   :3      NA's   :3
##      gad2      gad3      gad4      gad5
## Min.   :1.000   Min.   :1.000   Min.   :1.000   Min.   :1.000
## 1st Qu.:2.000   1st Qu.:2.000   1st Qu.:2.000   1st Qu.:2.000
## Median :2.000   Median :2.000   Median :2.000   Median :2.000
## Mean   :2.235   Mean   :2.176   Mean   :2.118   Mean   :2.222
## 3rd Qu.:3.000   3rd Qu.:3.000   3rd Qu.:2.000   3rd Qu.:3.000
## Max.   :4.000   Max.   :4.000   Max.   :4.000   Max.   :4.000
## NA's   :3      NA's   :3      NA's   :3      NA's   :2
##      gad6      gad7
## Min.   :1.000   Min.   :1.000
## 1st Qu.:2.000   1st Qu.:2.000
## Median :2.000   Median :2.000
## Mean   :2.167   Mean   :2.111
## 3rd Qu.:2.750   3rd Qu.:2.750
## Max.   :4.000   Max.   :3.000
## NA's   :2      NA's   :2
```

```
# you can also do this variable by variable in the df using the $
summary(NAdf$mark)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
##      55.00  64.25   69.50   70.00  74.75   90.00      4
```

This function automatically produces the min, quartiles, median, mean, max and count of NAs (if present) for all variables in the dataset. It also handles `mean()` without having to use the argument `na.rm` like we did before.

4.5.4 Removing missing data

There are a couple of methods of handling missingness.

4.5.4.1 Option A – ignore NAs inside a function Most tidyverse functions have: `na.rm = TRUE =` “remove NAs before calculating” which we used before to compute the mean.

```
# compute the mean ignoring the NAs
mean(NAdf$mark, na.rm = TRUE)
```

```
## [1] 70
```

This means you do not need to change the data at all, which is often preferred.

4.5.4.2 Option B – remove rows with any NA During data cleaning you can tell R to delete *ALL* rows with NAs in them. This is called **listwise deletion**.

```
# delete all rows with NA
df_clean <- NAdf %>% # make a new re-named df
  drop_na() # function is drop_na

# NB this is a case when it's really important to not overwrite your data!

# check it's length now (number of rows) compared to before
nrow(NAdf)
```

```
## [1] 20
```

```
nrow(df_clean)
```

```
## [1] 3
```

So, because we are deleting the ENTIRE row *when there is at least one NA* we have gone down from 20 observations (rows) to 3! This is a massive loss of data which is an issue. Especially if the missingness is systematic (e.g. a specific type of participant was more likely not to respond to a question). This can cause bias in your data.

4.5.5 Option C – remove NAs only for specific variables

It may be the case that you don't mind about NAs in all variables. Some variables aren't used in analysis for instance, so it would be silly to remove the entire observation. In this case you can tell R you only want to delete observations with NAs in specific variables (like mark and attendance)

```
# re-make the clean df with NA removed from 2 variables
df_clean <- NAdf %>%
  drop_na(mark, attendance)

# check it's length now (number of rows) compared to before
nrow(df_clean)
```

```
## [1] 13
```

The rationale behind these decisions, and when and why we would want to do them we will discuss in future more advanced statistics courses. The consequences of your decisions matter depending on how random or not the missingness is.

You will learn more about this in statistics modules, but briefly to finish this section.

- 1) **MCAR** – **Missing Completely At Random** - totally unrelated to anything - e.g., random internet dropout
- 2) **MAR** – **Missing At Random** - related to other variables - e.g., younger participants skip income
- 3) **MNAR** – **Missing Not At Random** - related to the value itself - e.g., highly anxious people avoid anxiety items - More problematic!

The more systematic the missingness, the more careful we must be.

4.5.6 Why shouldn't we swap NAs for 0's?

A very common mistake in data cleaning is: "I have missing values → I'll just change them to 0 so R can run the analysis." This is almost always statistically wrong and conceptually dangerous.

NA is not the same as 0!

- NA means "we do not know the value."
- 0 means "the value is genuinely zero."

These are completely different statements!

For example, take our GAD-7 scores in the NAdf. If we have two people, person A skipped the question because they were rushing and it was accidental, the other person didn't answer it because they had high anxiety. As a person collecting this data, you don't know the 'true situation' so NA correctly represents the situation as 'unknown'. If you re-code both to 0 you would risk making them both 'no anxiety' when this is only true of one participant, not both.

Person	True situation	What NA means	What 0 would mean
A	Didn't answer item due to rushing	Unknown	"Perfect mental health"
B	Skipped question due to anxiety	Unknown	"Perfect mental health"

By converting NA → 0 you are inventing extremely positive scores that were never observed.

4.5.6.1 0 bias means down, NAs don't Imagine you have 3 scores, and one missing value. If handled correctly the mean is:

```
# Imagine 3 real scores and one missing
scores <- c(12, 14, 16, NA)

mean(scores, na.rm = TRUE)
```

```
## [1] 14
```

but if you changed that last value to a 0 then it becomes a completely different value, and tells a very different story about the sample.

```
mean(c(12, 14, 16, 0))
```

```
## [1] 10.5
```

This actually happens in science and leads big papers being retracted. For those interested have a look at Beheim et al. (2021) Nature article. This is research on big gods and social complexity.

The original dataset contained many missing observations for if small scale societies believed in big gods or not. The analysis didn't handle them correctly, coding them as 0 - big gods were absent - rather than NAs. This artificially:

- inflated the numbers of societies with 'no big gods'
- strengthened the relationship between religion and social complexity but incorrectly.

When the authors re-analysed the data treating NAs correctly the original published findings no longer held and the paper was retracted!

So don't change NAs to 0!

5 Data Visualisation in R: Week 5

5.1 Summarising data

5.2 Plotting with ggplot2

There are a number of packages to make graphs but ggplot2 is the most versatile and widely used. Let's make sure its installed and loaded.

```
library(ggplot2)
```

5.2.1 Iris flower data frame

To start learning the grammar of graphics we are going to use a simple dataframe - Fisher's Iris flower dataset. The data set consists of 50 samples from each of three species of Iris (*Iris setosa*, *Iris virginica* and *Iris versicolor*). Four features were measured from each sample: the length and the width of the sepals and petals, in centimetres.

Let's have a look at this quickly.

```
iris # run this line to view the df - as it is installed in base R nothing else is required
```

##	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
## 1	5.1	3.5	1.4	0.2	setosa
## 2	4.9	3.0	1.4	0.2	setosa
## 3	4.7	3.2	1.3	0.2	setosa
## 4	4.6	3.1	1.5	0.2	setosa
## 5	5.0	3.6	1.4	0.2	setosa
## 6	5.4	3.9	1.7	0.4	setosa
## 7	4.6	3.4	1.4	0.3	setosa
## 8	5.0	3.4	1.5	0.2	setosa
## 9	4.4	2.9	1.4	0.2	setosa
## 10	4.9	3.1	1.5	0.1	setosa
## 11	5.4	3.7	1.5	0.2	setosa
## 12	4.8	3.4	1.6	0.2	setosa
## 13	4.8	3.0	1.4	0.1	setosa

## 14	4.3	3.0	1.1	0.1	setosa
## 15	5.8	4.0	1.2	0.2	setosa
## 16	5.7	4.4	1.5	0.4	setosa
## 17	5.4	3.9	1.3	0.4	setosa
## 18	5.1	3.5	1.4	0.3	setosa
## 19	5.7	3.8	1.7	0.3	setosa
## 20	5.1	3.8	1.5	0.3	setosa
## 21	5.4	3.4	1.7	0.2	setosa
## 22	5.1	3.7	1.5	0.4	setosa
## 23	4.6	3.6	1.0	0.2	setosa
## 24	5.1	3.3	1.7	0.5	setosa
## 25	4.8	3.4	1.9	0.2	setosa
## 26	5.0	3.0	1.6	0.2	setosa
## 27	5.0	3.4	1.6	0.4	setosa
## 28	5.2	3.5	1.5	0.2	setosa
## 29	5.2	3.4	1.4	0.2	setosa
## 30	4.7	3.2	1.6	0.2	setosa
## 31	4.8	3.1	1.6	0.2	setosa
## 32	5.4	3.4	1.5	0.4	setosa
## 33	5.2	4.1	1.5	0.1	setosa
## 34	5.5	4.2	1.4	0.2	setosa
## 35	4.9	3.1	1.5	0.2	setosa
## 36	5.0	3.2	1.2	0.2	setosa
## 37	5.5	3.5	1.3	0.2	setosa
## 38	4.9	3.6	1.4	0.1	setosa
## 39	4.4	3.0	1.3	0.2	setosa
## 40	5.1	3.4	1.5	0.2	setosa
## 41	5.0	3.5	1.3	0.3	setosa
## 42	4.5	2.3	1.3	0.3	setosa
## 43	4.4	3.2	1.3	0.2	setosa
## 44	5.0	3.5	1.6	0.6	setosa
## 45	5.1	3.8	1.9	0.4	setosa
## 46	4.8	3.0	1.4	0.3	setosa
## 47	5.1	3.8	1.6	0.2	setosa
## 48	4.6	3.2	1.4	0.2	setosa
## 49	5.3	3.7	1.5	0.2	setosa
## 50	5.0	3.3	1.4	0.2	setosa
## 51	7.0	3.2	4.7	1.4	versicolor
## 52	6.4	3.2	4.5	1.5	versicolor
## 53	6.9	3.1	4.9	1.5	versicolor
## 54	5.5	2.3	4.0	1.3	versicolor
## 55	6.5	2.8	4.6	1.5	versicolor
## 56	5.7	2.8	4.5	1.3	versicolor
## 57	6.3	3.3	4.7	1.6	versicolor
## 58	4.9	2.4	3.3	1.0	versicolor
## 59	6.6	2.9	4.6	1.3	versicolor
## 60	5.2	2.7	3.9	1.4	versicolor
## 61	5.0	2.0	3.5	1.0	versicolor
## 62	5.9	3.0	4.2	1.5	versicolor
## 63	6.0	2.2	4.0	1.0	versicolor
## 64	6.1	2.9	4.7	1.4	versicolor
## 65	5.6	2.9	3.6	1.3	versicolor
## 66	6.7	3.1	4.4	1.4	versicolor
## 67	5.6	3.0	4.5	1.5	versicolor

## 68	5.8	2.7	4.1	1.0 versicolor
## 69	6.2	2.2	4.5	1.5 versicolor
## 70	5.6	2.5	3.9	1.1 versicolor
## 71	5.9	3.2	4.8	1.8 versicolor
## 72	6.1	2.8	4.0	1.3 versicolor
## 73	6.3	2.5	4.9	1.5 versicolor
## 74	6.1	2.8	4.7	1.2 versicolor
## 75	6.4	2.9	4.3	1.3 versicolor
## 76	6.6	3.0	4.4	1.4 versicolor
## 77	6.8	2.8	4.8	1.4 versicolor
## 78	6.7	3.0	5.0	1.7 versicolor
## 79	6.0	2.9	4.5	1.5 versicolor
## 80	5.7	2.6	3.5	1.0 versicolor
## 81	5.5	2.4	3.8	1.1 versicolor
## 82	5.5	2.4	3.7	1.0 versicolor
## 83	5.8	2.7	3.9	1.2 versicolor
## 84	6.0	2.7	5.1	1.6 versicolor
## 85	5.4	3.0	4.5	1.5 versicolor
## 86	6.0	3.4	4.5	1.6 versicolor
## 87	6.7	3.1	4.7	1.5 versicolor
## 88	6.3	2.3	4.4	1.3 versicolor
## 89	5.6	3.0	4.1	1.3 versicolor
## 90	5.5	2.5	4.0	1.3 versicolor
## 91	5.5	2.6	4.4	1.2 versicolor
## 92	6.1	3.0	4.6	1.4 versicolor
## 93	5.8	2.6	4.0	1.2 versicolor
## 94	5.0	2.3	3.3	1.0 versicolor
## 95	5.6	2.7	4.2	1.3 versicolor
## 96	5.7	3.0	4.2	1.2 versicolor
## 97	5.7	2.9	4.2	1.3 versicolor
## 98	6.2	2.9	4.3	1.3 versicolor
## 99	5.1	2.5	3.0	1.1 versicolor
## 100	5.7	2.8	4.1	1.3 versicolor
## 101	6.3	3.3	6.0	2.5 virginica
## 102	5.8	2.7	5.1	1.9 virginica
## 103	7.1	3.0	5.9	2.1 virginica
## 104	6.3	2.9	5.6	1.8 virginica
## 105	6.5	3.0	5.8	2.2 virginica
## 106	7.6	3.0	6.6	2.1 virginica
## 107	4.9	2.5	4.5	1.7 virginica
## 108	7.3	2.9	6.3	1.8 virginica
## 109	6.7	2.5	5.8	1.8 virginica
## 110	7.2	3.6	6.1	2.5 virginica
## 111	6.5	3.2	5.1	2.0 virginica
## 112	6.4	2.7	5.3	1.9 virginica
## 113	6.8	3.0	5.5	2.1 virginica
## 114	5.7	2.5	5.0	2.0 virginica
## 115	5.8	2.8	5.1	2.4 virginica
## 116	6.4	3.2	5.3	2.3 virginica
## 117	6.5	3.0	5.5	1.8 virginica
## 118	7.7	3.8	6.7	2.2 virginica
## 119	7.7	2.6	6.9	2.3 virginica
## 120	6.0	2.2	5.0	1.5 virginica
## 121	6.9	3.2	5.7	2.3 virginica

## 122	5.6	2.8	4.9	2.0	virginica
## 123	7.7	2.8	6.7	2.0	virginica
## 124	6.3	2.7	4.9	1.8	virginica
## 125	6.7	3.3	5.7	2.1	virginica
## 126	7.2	3.2	6.0	1.8	virginica
## 127	6.2	2.8	4.8	1.8	virginica
## 128	6.1	3.0	4.9	1.8	virginica
## 129	6.4	2.8	5.6	2.1	virginica
## 130	7.2	3.0	5.8	1.6	virginica
## 131	7.4	2.8	6.1	1.9	virginica
## 132	7.9	3.8	6.4	2.0	virginica
## 133	6.4	2.8	5.6	2.2	virginica
## 134	6.3	2.8	5.1	1.5	virginica
## 135	6.1	2.6	5.6	1.4	virginica
## 136	7.7	3.0	6.1	2.3	virginica
## 137	6.3	3.4	5.6	2.4	virginica
## 138	6.4	3.1	5.5	1.8	virginica
## 139	6.0	3.0	4.8	1.8	virginica
## 140	6.9	3.1	5.4	2.1	virginica
## 141	6.7	3.1	5.6	2.4	virginica
## 142	6.9	3.1	5.1	2.3	virginica
## 143	5.8	2.7	5.1	1.9	virginica
## 144	6.8	3.2	5.9	2.3	virginica
## 145	6.7	3.3	5.7	2.5	virginica
## 146	6.7	3.0	5.2	2.3	virginica
## 147	6.3	2.5	5.0	1.9	virginica
## 148	6.5	3.0	5.2	2.0	virginica
## 149	6.2	3.4	5.4	2.3	virginica
## 150	5.9	3.0	5.1	1.8	virginica

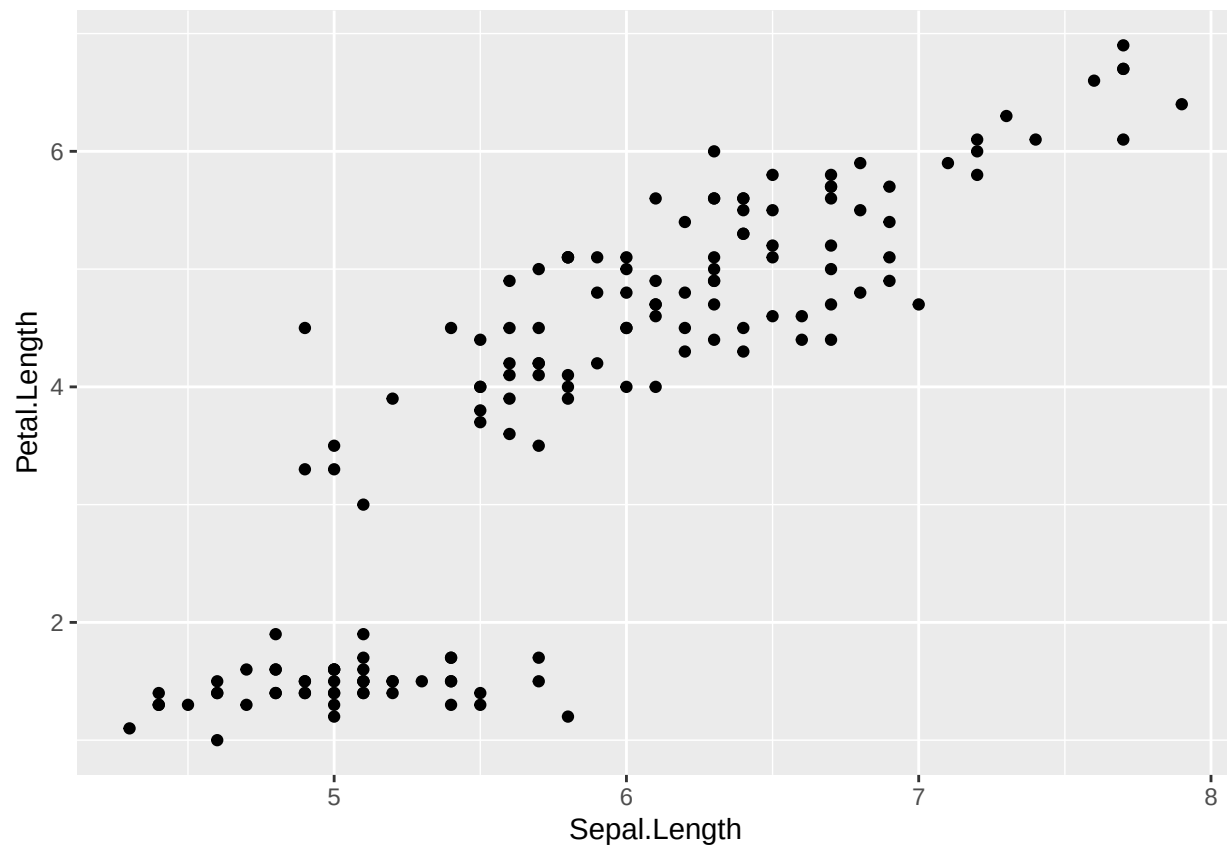
5.2.2 Exploration of Iris dataset

We are interested in understanding the **relationship between Sepal length and Petal length** - are they correlated? Is this a negative or positive correlation?

The best way to look at this is in a scatterplot with all the data points plotted by sepal length (on the x axis, bottom) and by petal length (on the y axis, left)

```
# Scatterplot using ggplot

ggplot(data = iris) + # the + means R is waiting for more info
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length))
```



You have a scatterplot!

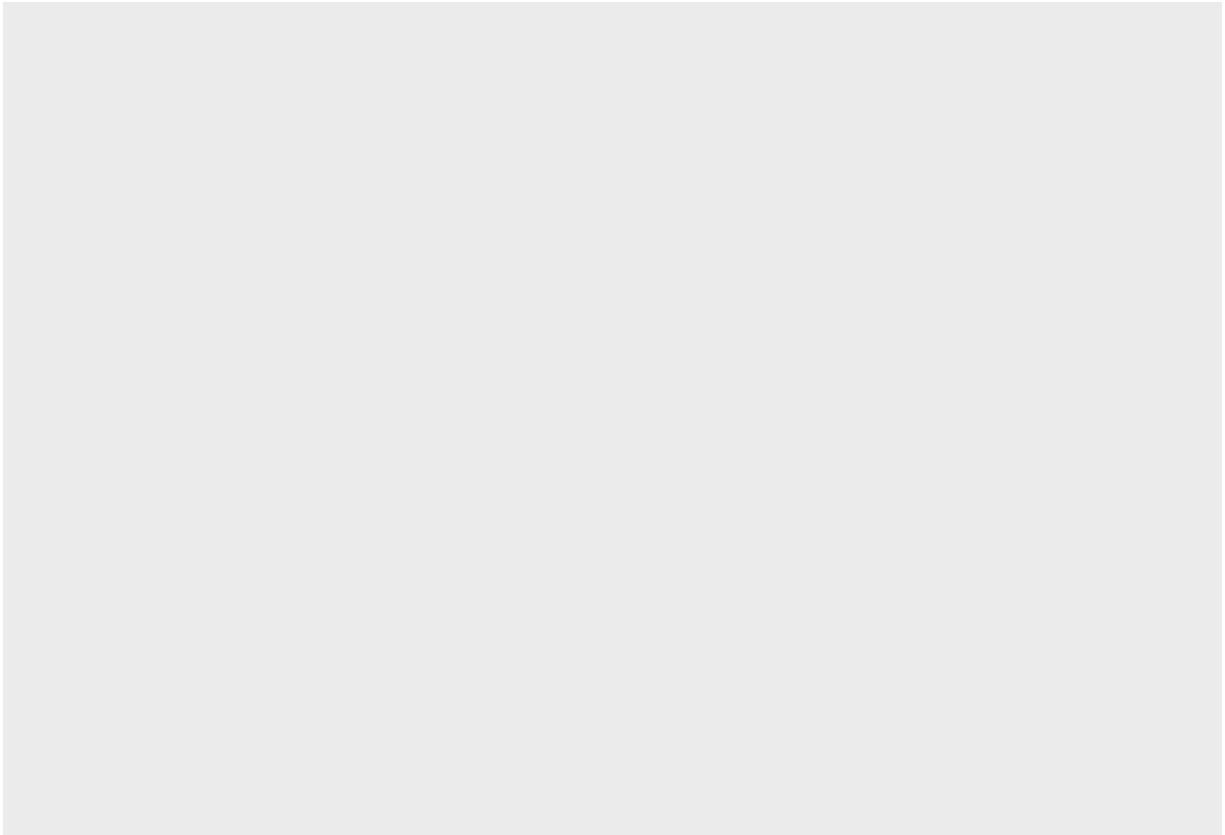
In the above code `ggplot()` is the function which calls the plot (specifically the coordinate system you add layers to), and the object is the `iris` dataframe. This is followed by a second function `geom_point()` which calls the *dots* making the plot a scatter plot. its like saying, I want a plot, and then I want this plot to be a scatterplot.

FYI, as we will soon find out there are tons of `geom_` functions dependent on the type of plot you want to make. As were are interested in the relationship between two continuous variables a scatterplot makes sense.

After the functions comes the *arguments* - the content of the brackets. The first argument of `ggplot()` is the dataset to use in the graph. So `ggplot(data = iris)` creates an empty graph (if you didn't add the layer you would have blank panel). You can try this:

```
# just run the ggplot without the scatterplot layer
```

```
ggplot(data = iris)
```



You have nothing!

The `geom_point(mapping = aes())` argument defines how variables in your dataset are plotted. The mapping argument is always paired with `aes()`, and the `x` and `y` arguments of `aes()` specify which variables to map to the x (sepal.length) and y (petal.length) axes. This is what is required to see something in your plot. You are basically saying:

1: I want a plot please based on the iris data -> `ggplot(data = iris)` 2: I want this plot to be a scatterplot with points -> `geom_point()` 3: I want this plot to be of sepal length (X) and petal length (y) -> `(mapping = aes(x = Sepal.Length, y = Petal.Length))`

That's it! You can change this template by change the **data**, **geom_function** or **mapping variables** to create a wide range of graphics using different data.

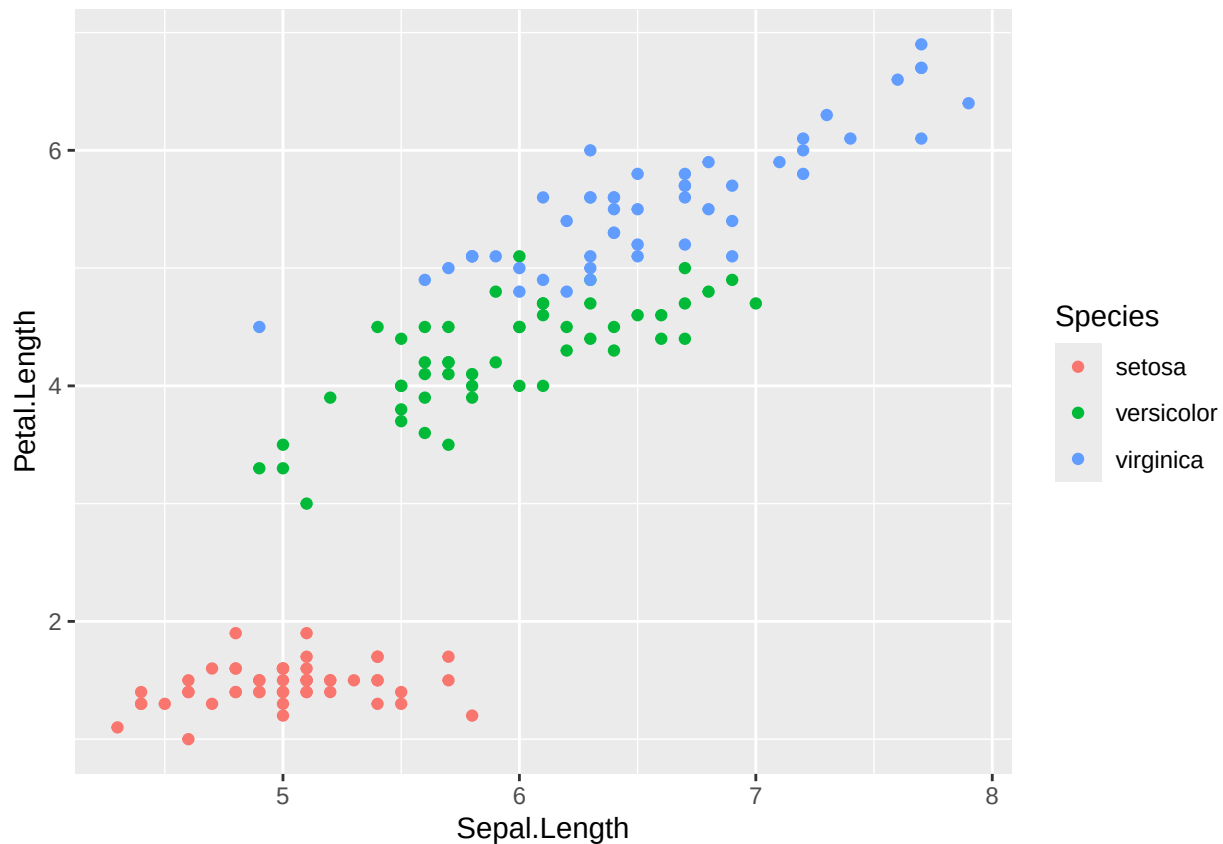
You can see that ggplot2 is a lot about layering. You make a basic plot and then you add features to help you better understand it. So let's take this plot further. It looks like there is a positive relationship between sepal and petal length, *however there is a bunch of data points in the far left corner which this does hold for - why is this? how can we explore this further?*

5.2.3 Adding aesthetics

To better visualise our data we can use aesthetics by adding a third variable to the mapping argument (the bit in the `aes()`). An aesthetic is a visual property of the objects in your plot, which includes colour, size and shape of the data points. You are able to change these elements, both globally (i.e., for all points on the graph) and per group (i.e., different colours are attributed to different plant species).

Here we can use colour to map aesthetics in your plot to the variables in the iris dataset - let's do species to see if that helps us understand the data better.

```
# Add in a colour = Species into the argument
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length, colour = Species))
```



```
# note colour can be spelt both in UK and US English!
```

The `colour = Species` argument has coloured the dots either red, green or blue depending on the species label. This is called scaling. Ggplot has also automatically given you a legend on the right-hand side to explain which species is which colour.

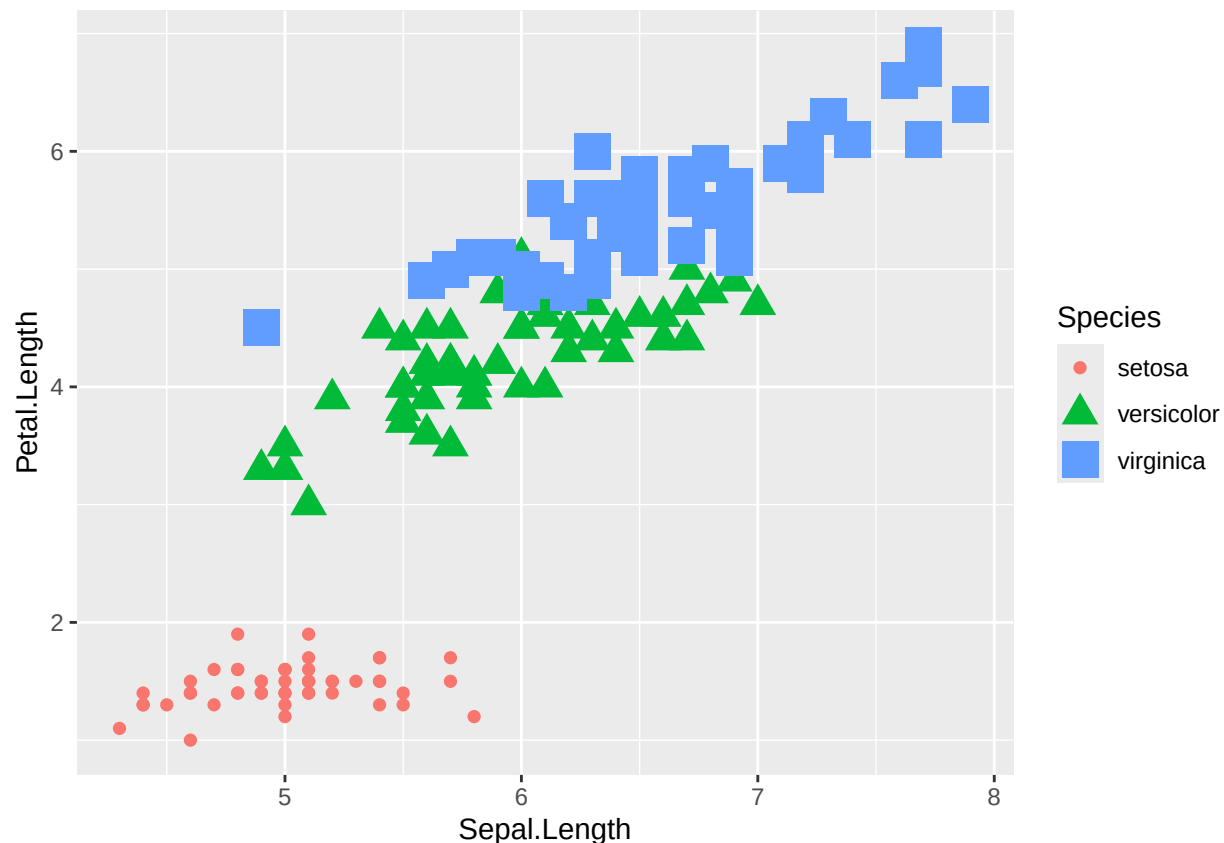
Note that the text has remained the same but I have just added in the `colour` argument at the end of the `aes()` brackets. This is using the variable `Species` from the dataset to colour the data points depending on their species label.

Now we can see that the linear relationship between sepal and petal length holds true for `versicolor` and `virginica` but not for `setosa`! The red points are flat.

Let's now try other aesthetics like the shape type or the shape size

```
# include shape = Species and size = Species
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species, shape = Species, size = Species))
```

```
## Warning: Using size for a discrete variable is not advised.
```



```
# can keep adding arguments into the aes()
```

So now we have added in size and shape arguments. Each of the different species are associated with different sizes of points and different shapes. R is giving us a warning that size is not advised for a discrete variable because it is an unordered variable.

Basically, the issue is that bigger points makes you think some species are bigger than others. This isn't the case so it's confusing and doesn't help understanding the data - I would remove this.

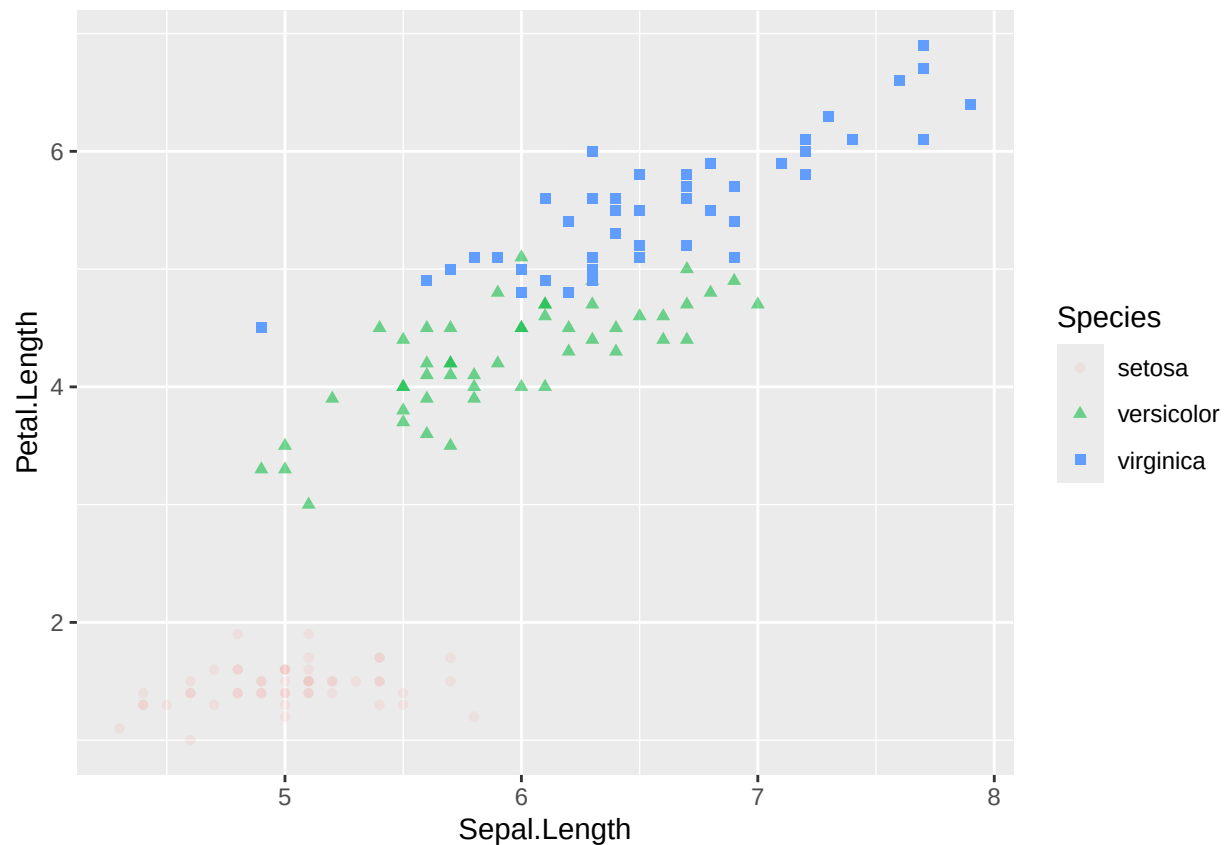
Also note that all three of these aesthetics have been included in the legend!

So instead of changing the size we could try changing the transparency of the points using the alpha argument.

```
# Add alpha = Species to the plot
```

```
ggplot(data = iris) +  
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,  
                           color = Species, shape = Species, alpha = Species))
```

```
## Warning: Using alpha for a discrete variable is not advised.
```



Now setosa points are hardly visible - not really useful. This is just to show you can change all these elements, but much is not necessarily better and plots should be easy to understand! Plots should communicate something useful about the data. Again, I would just remove this.

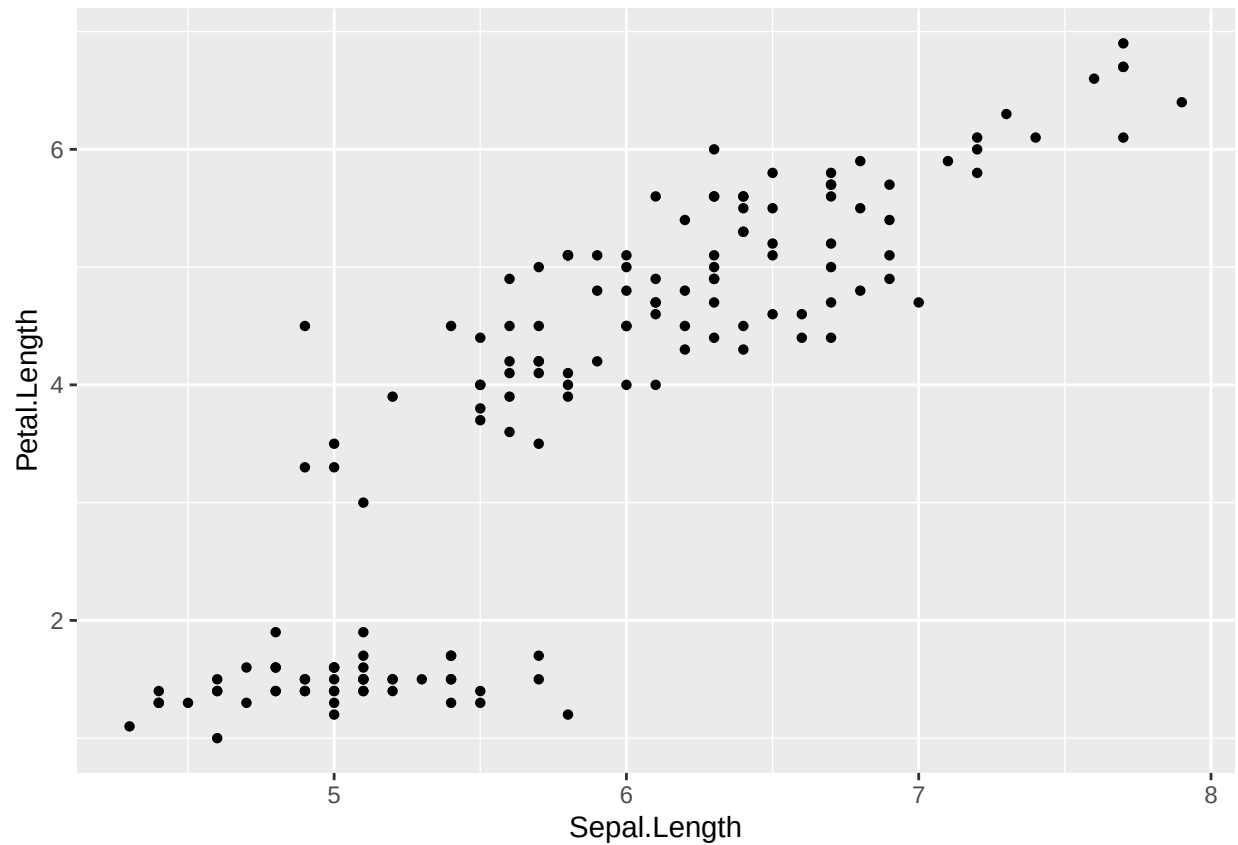
You can also set the aesthetic properties of your geom manually just outside the `aes()`.

For example, we can make all of the points in our plot pink, all the shapes circles or all the points larger. When you do this the global aesthetic change will overwrite anything in the mapping argument -

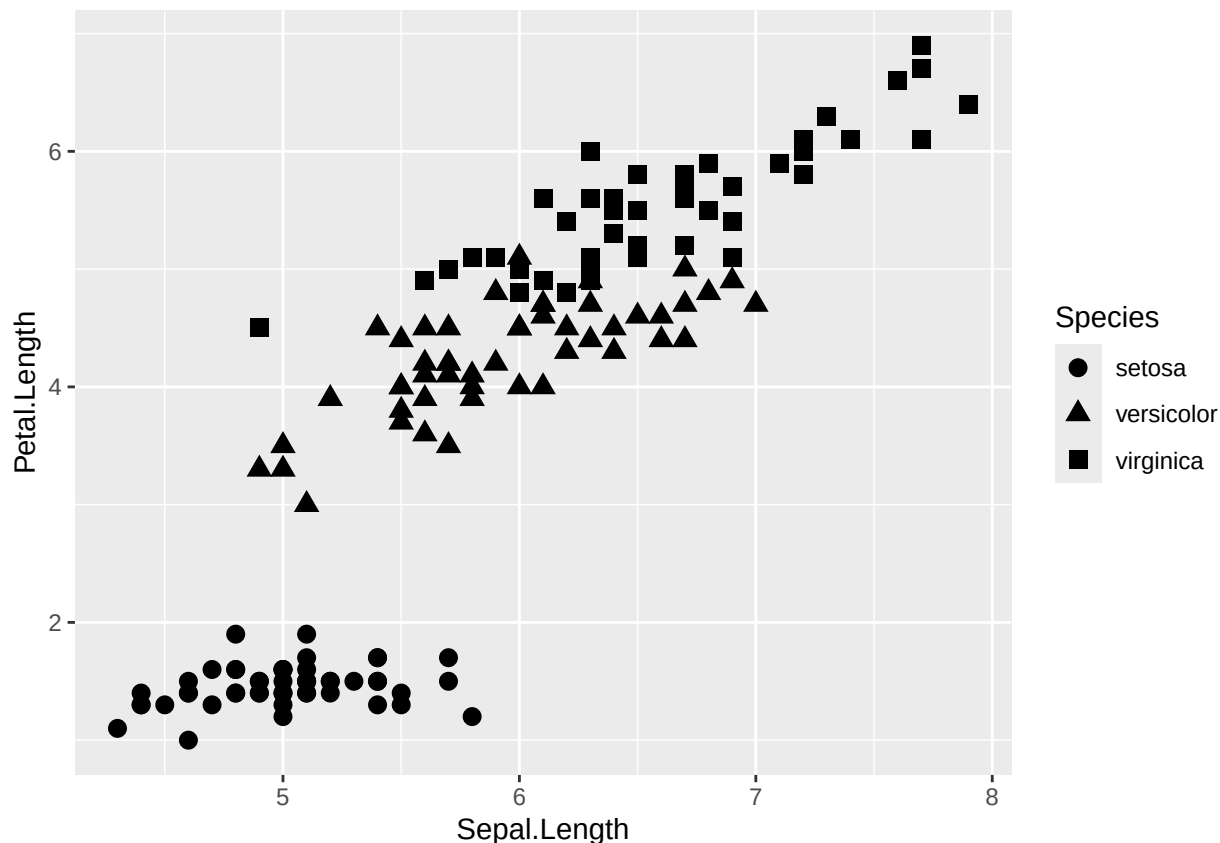
```
# colour - all pink
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           shape = Species), colour = "salmon")
```



```
# shape
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
    shape = Species), shape = 16)
```



```
# 16 = small circles.  
# Note that this has overrode the shape = species in the mapping argument.  
  
# size  
ggplot(data = iris) +  
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,  
                           shape = Species), size = 3)
```



```
# bigger number = bigger points
```

There is an amazing range of colours you can use in R: See [Colours in R quick reference](#) for colour names (replace the “salmon” above).

R has 25 built-in shapes which are identified by numbers. Shapes change, as do their colour and fill. You can see the different colours to call in the argument [here](#)

You can see further information here in the online version of [cookbook for R](#). See: [Shapes and Line Types in R](#)

Because these changes to the plot are happening outside of the `aes()` they do not convey any information about the variable so generally less informative (but pretty!)

5.2.4 Adding layers to your plot

From the selection of plots above I think the most useful is the first one with the colours denoted by species. Often scatter graphs come with lines to show the trends in the data. This can be done using a different *geom_function*

geom are geometrical objects used to represent data and ggplot has over 40 of them. So the one you pick depends on the data type you want to represent and graph you want to make. So to add a line to a your existing scatterplot you need to use `geom_smooth()`, which is a smooth line fitted to the data.

You can see why it's called a layer, because you just keep adding `geom_` to each other as you build up your plot.

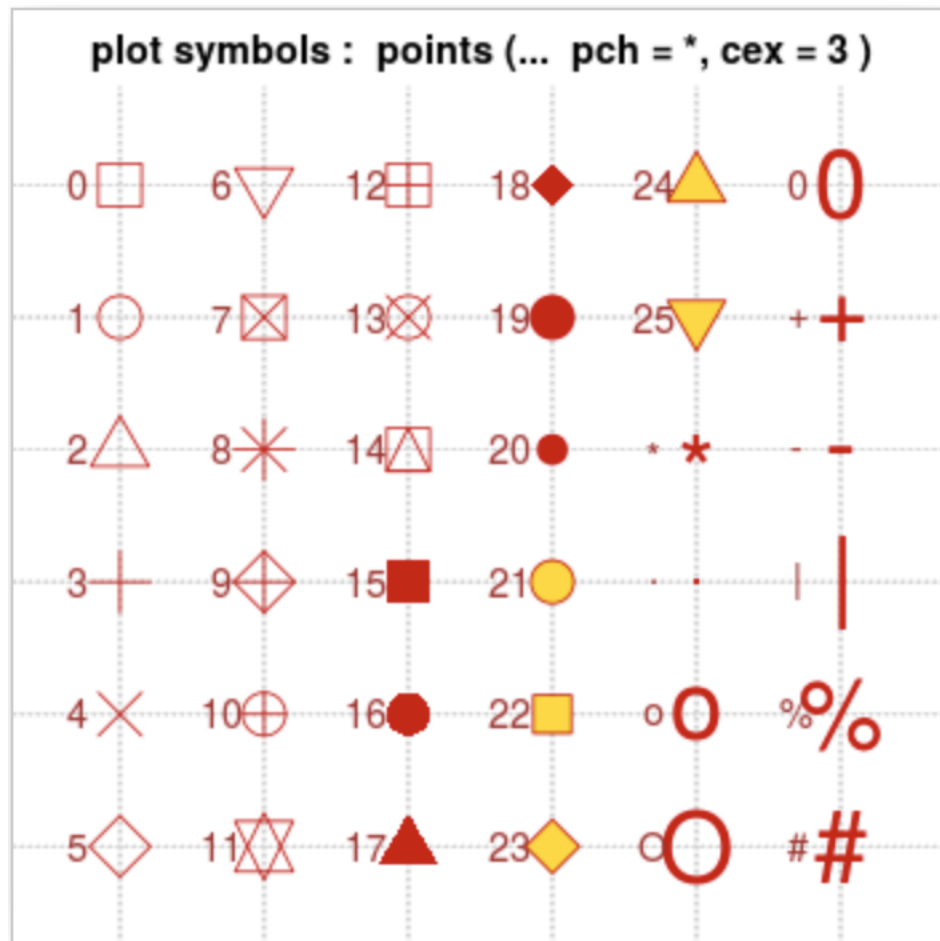
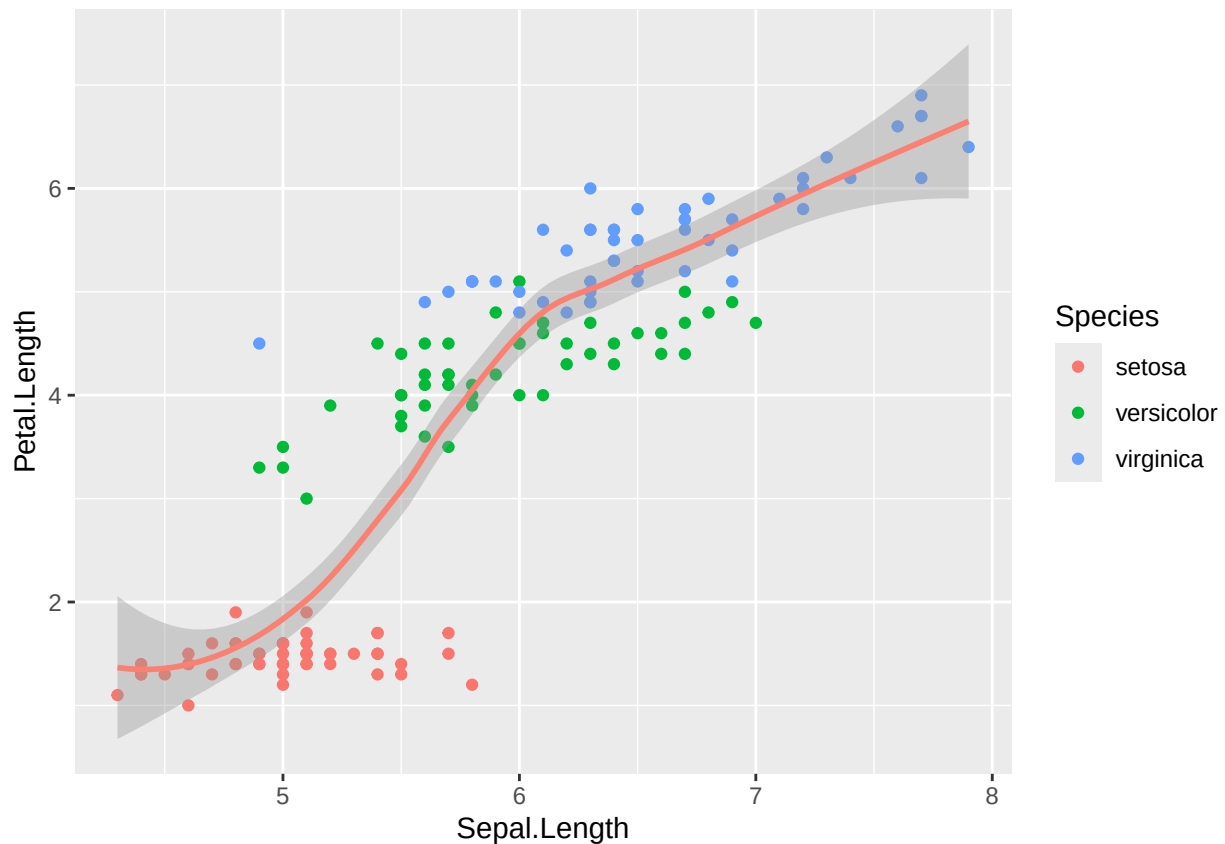


Figure 8: *Figure 5.1 Shape types and their numbers. From cookbook for R*

```
# Add a line with 95% confidence intervals
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species)) + # same as above AND....
  geom_smooth(mapping = aes(x = Sepal.Length, y = Petal.Length), colour = "salmon")
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```

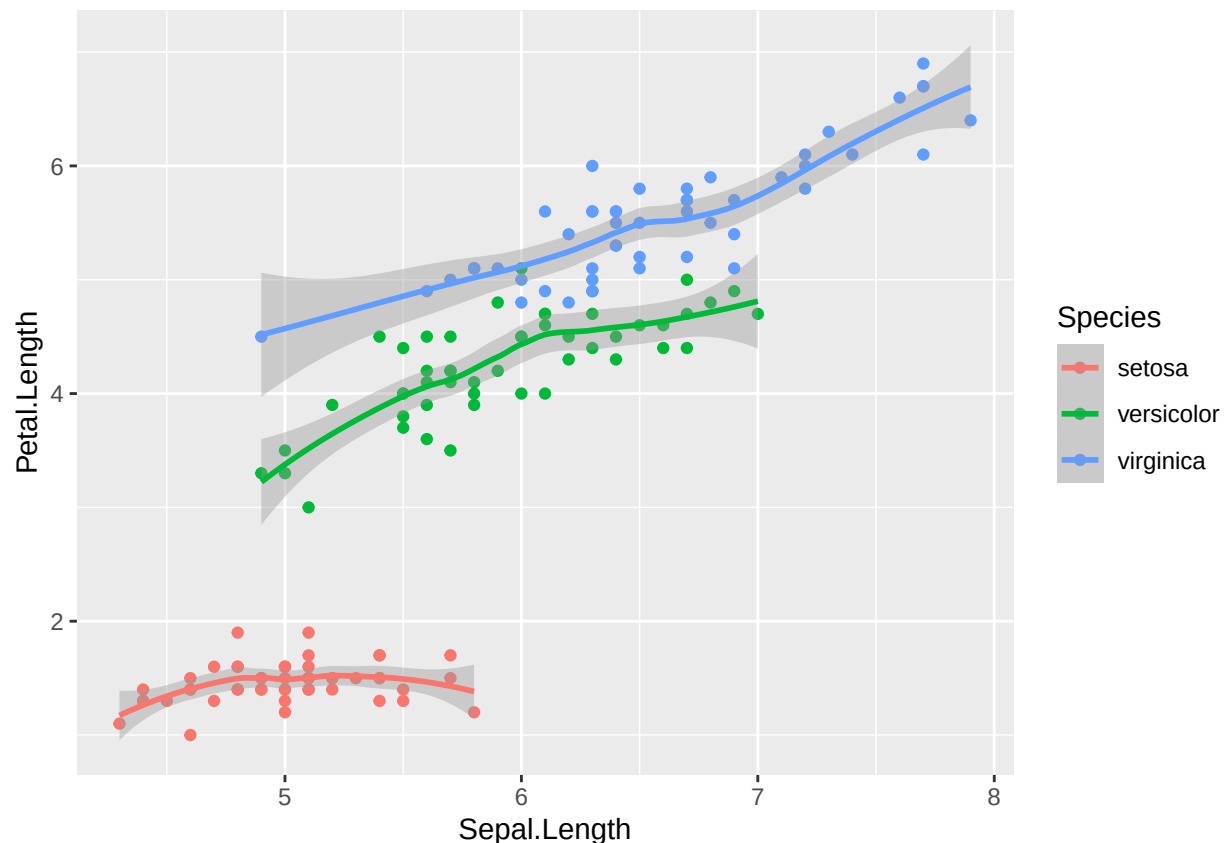


```
# adds smoothed line between sepal and petal length which is Salmon colour
```

So we have added a smooth line with a 95% confidence region (as R tells you above by the info icon), and using the global aesthetic made it salmon coloured. Nice. But, if we know the relationship between sepal length and petal length is dependent on the species, it is probably better to have a line for each of these species. We do this in exactly the same way as we changed the aesthetics on the points *by adding a third condition into the mapping argument*.

```
# line for each of the species
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species)) + # same as above AND
  geom_smooth(mapping = aes(x = Sepal.Length, y = Petal.Length, colour = Species))
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



```
# line is now by species as the colour = Species is in the aes()
```

So now you can see a smoothed fit curve for each of the species. So this shows how you can add on layers to the basic graphic and how each geom_function can be changed using the mapping function.

It is important to note that you cannot use the same arguments for all the geom types. For instance, you can't change the line 'shape' like a data point, but you can change the linetype, and make that specific to each species.

```
# line for each of the species
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species)) + # same as above and the ADD
  geom_smooth(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           colour = Species, linetype = Species))
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```

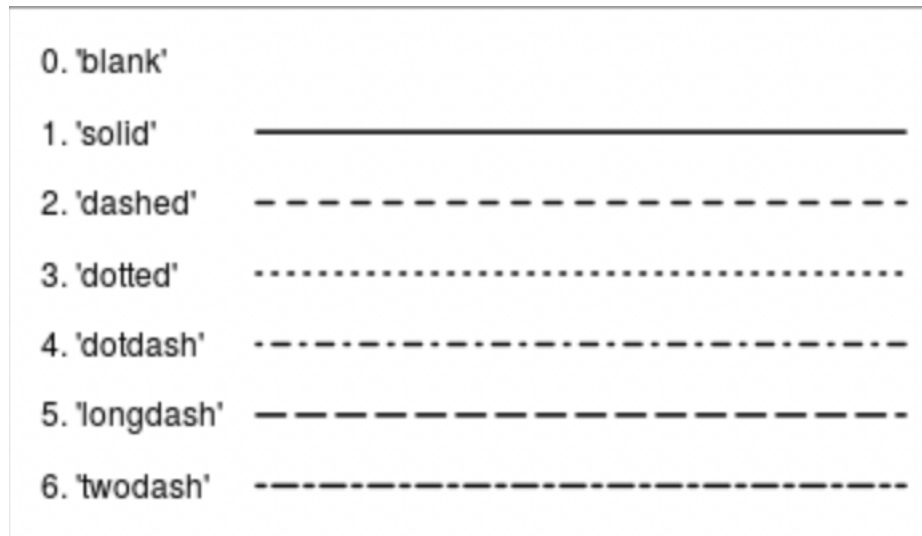
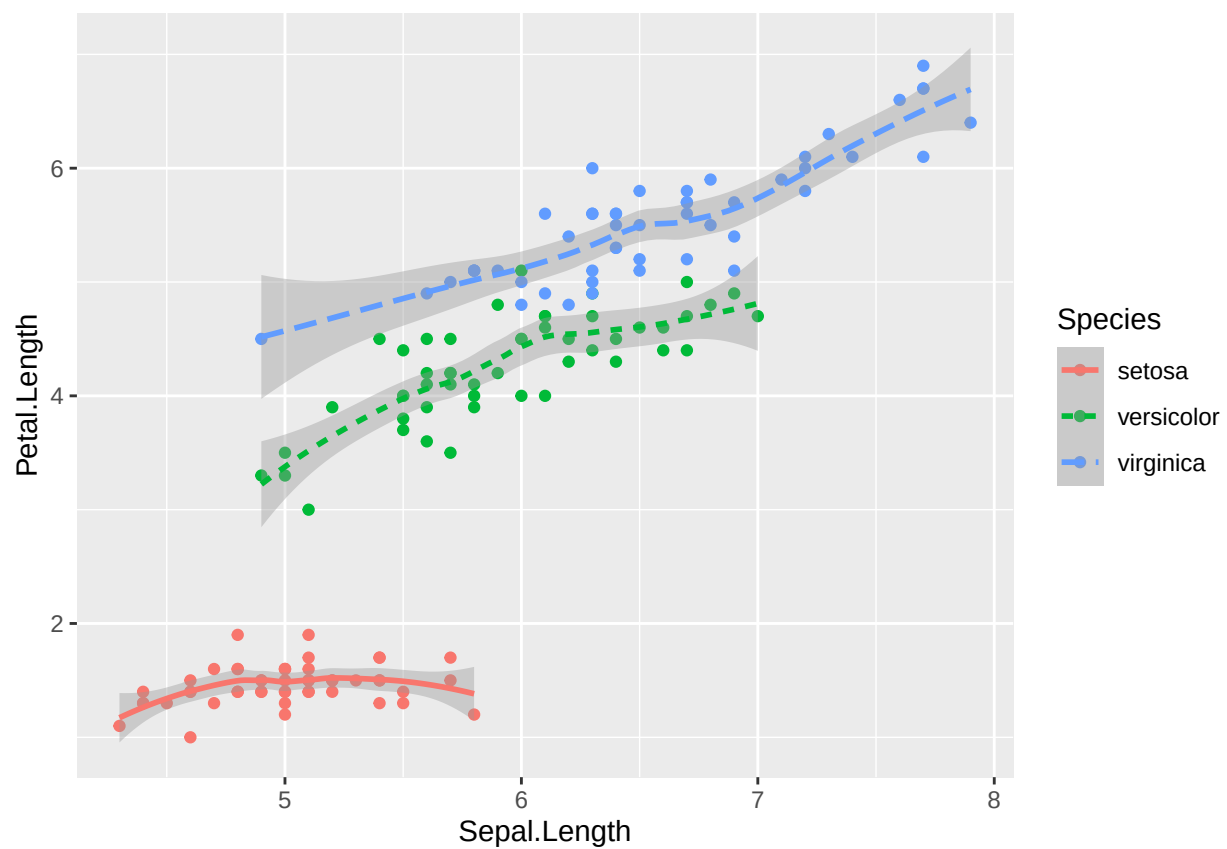


Figure 9: *Figure 5.2 line types and their numbers. From cookbook for R*



```
# change the linetype by species
```

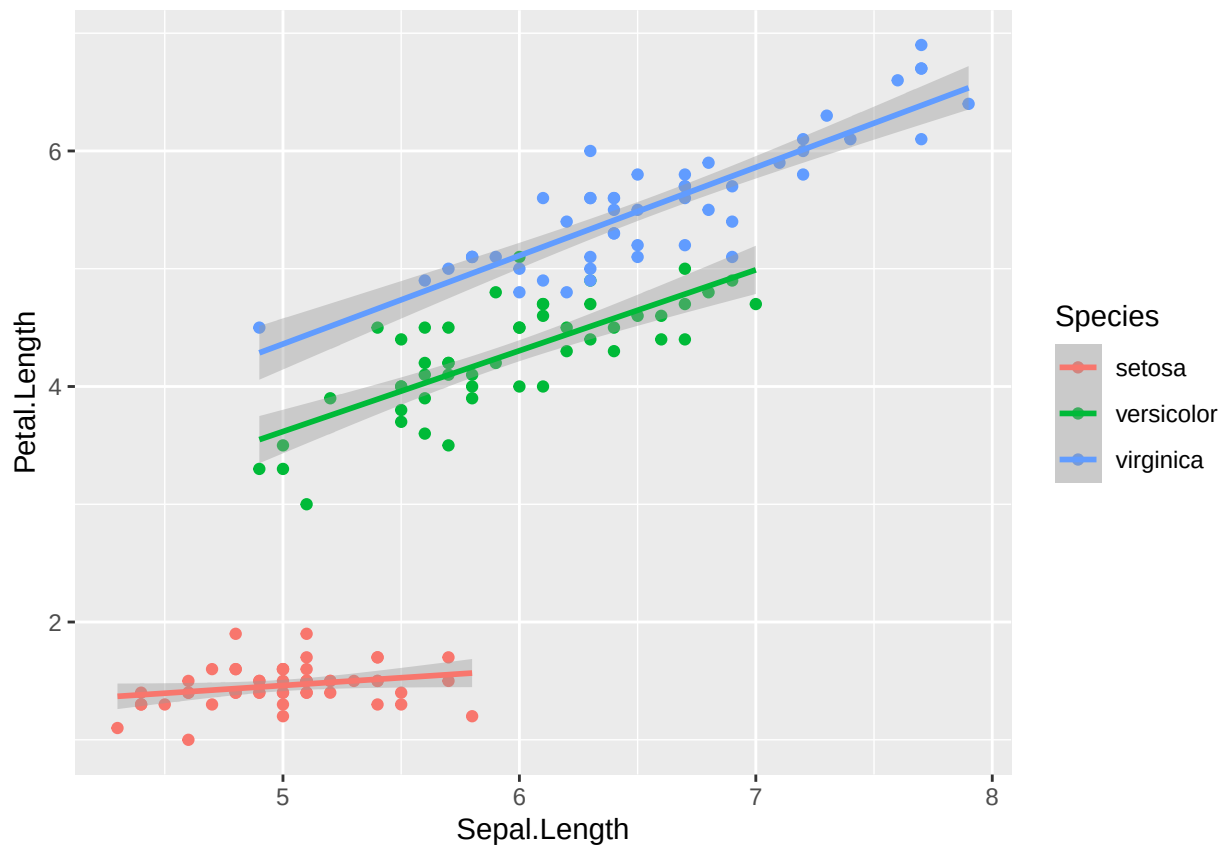
This feature doesn't add much to our current plot because we changed the colour, but very useful if the plot was black and white as we have solid and dashed lines.

Finally, you can change the line types between a smooth curve or a regression line, and whether it has 95%

CI or not.

```
# linear regression lines with 95% CI
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species)) +
  geom_smooth(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           colour = Species), method = lm)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



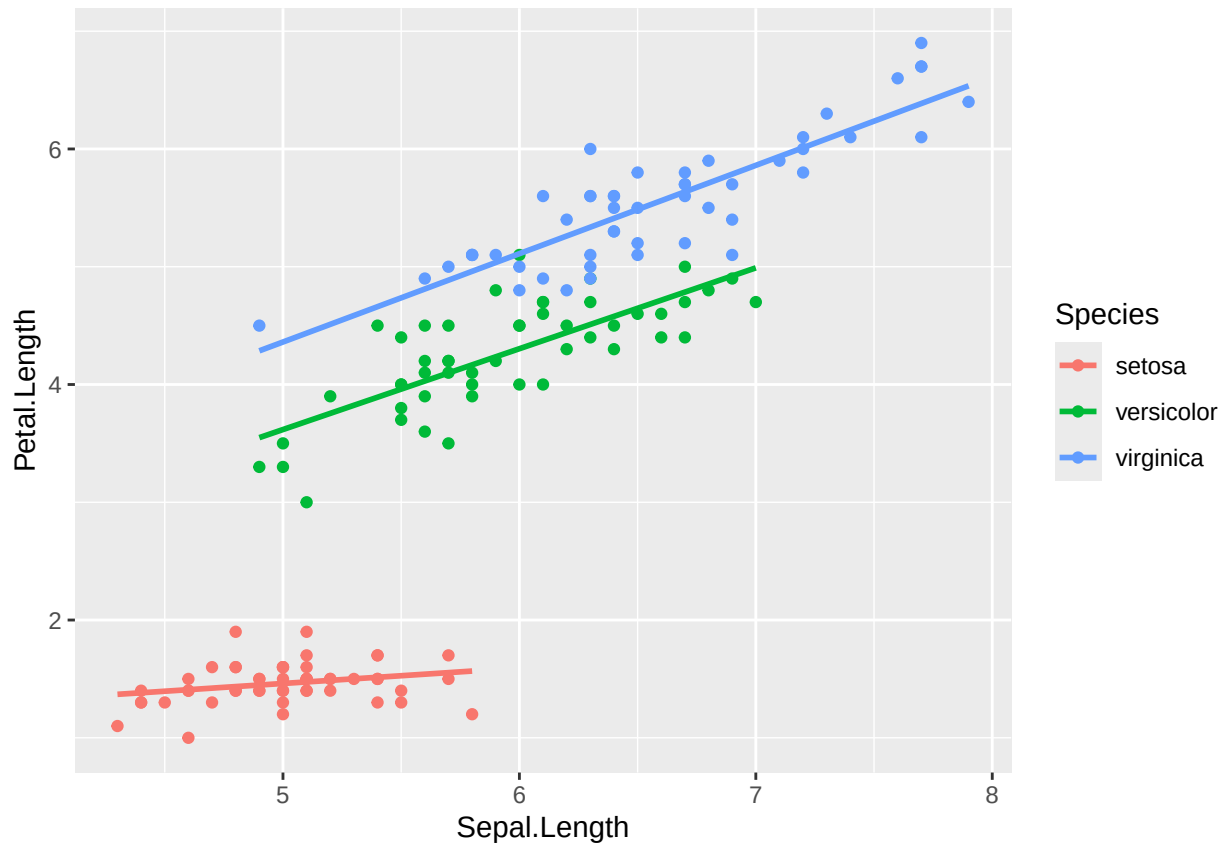
```
# add linear regression line with the argument method=lm
# 95% confidence intervals are included as default
```

We can now see each species has its own colour and three regression (line of best fit) lines which is included in the legend.

We can also turn off the 95% CI if we want (but normally we want them)

```
# linear regression lines without 95% CI
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           color = Species)) +
  geom_smooth(mapping = aes(x = Sepal.Length, y = Petal.Length,
                           colour = Species), method = lm, se=FALSE)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
# SE=FLASE removes the 95% CI
```

5.2.5 Streamlining your code

Our code is working well, we can see the trends for the three species clearly. However, it's a bit long winded - more space to go wrong, especially if we add in more layers.

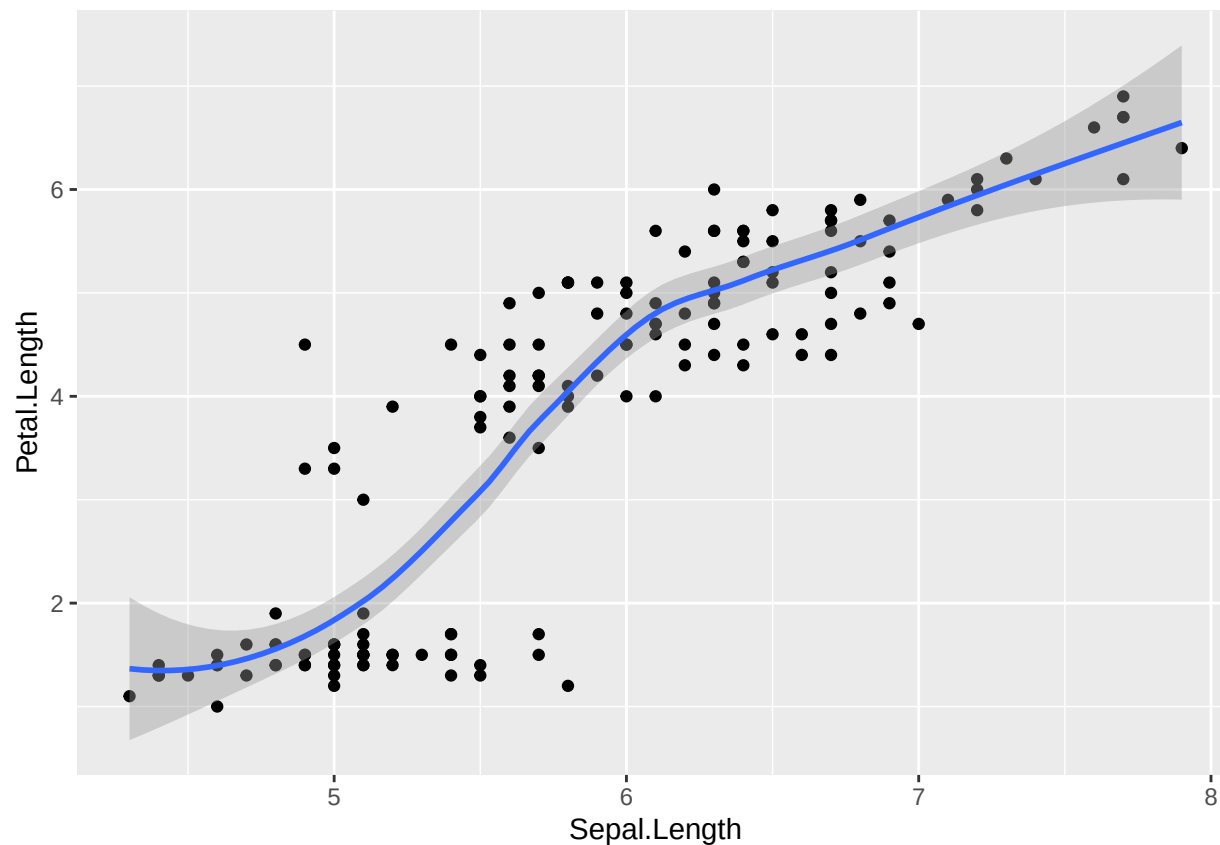
For instance, if you want to change the x and y variable you need to do this in both lines of code.

It is always good to make your code as efficient as possible. We don't need to repeat the mapping for both `geom_` functions if we state it in the `ggplot()` function, `ggplot2` then treats these mappings as global mappings and applies it to both geom in the graph.

This saves use from repeating

```
# move the x and y to the ggplot function  
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +  
  geom_point() +  
  geom_smooth()
```

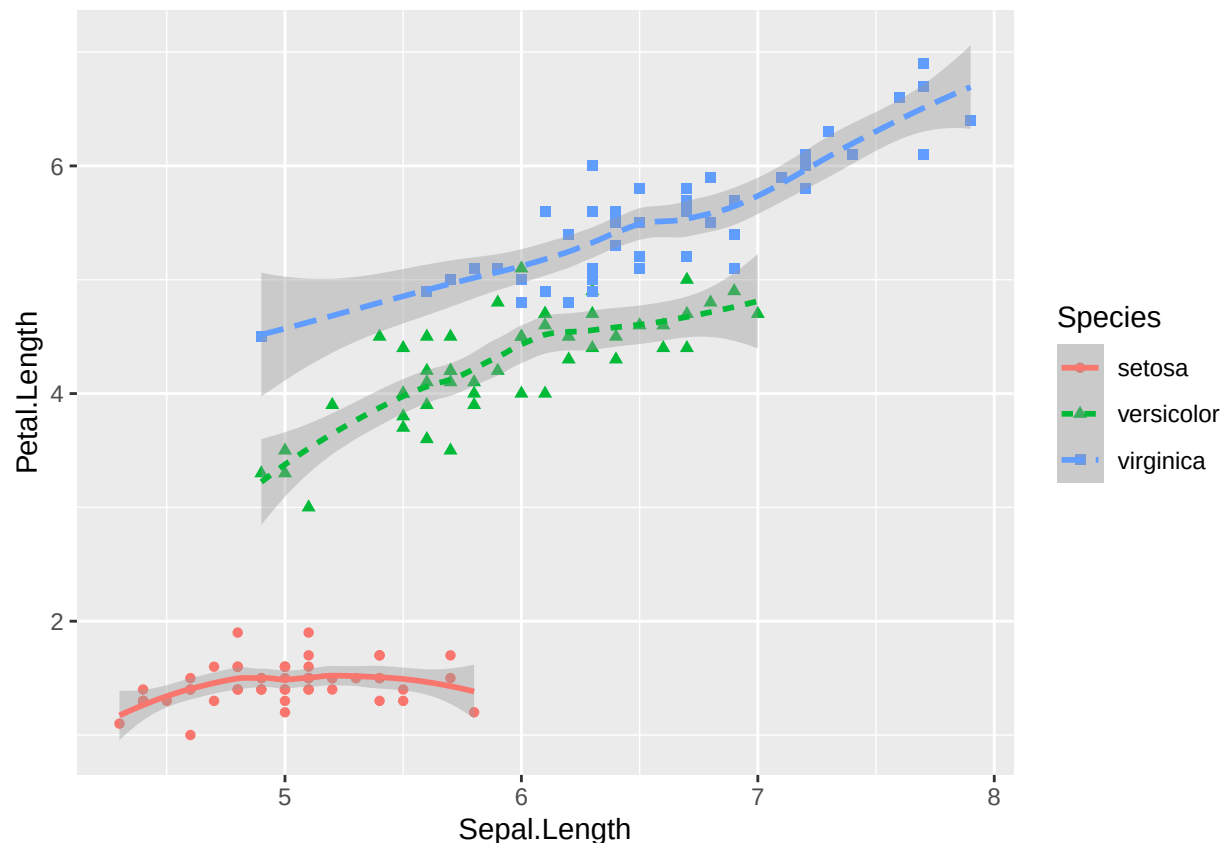
```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



If you place mappings in a geom function, ggplot2 will treat them as local mappings for the layer. It will use these mappings to extend or overwrite the global mappings for that layer only. This makes it possible to display different aesthetics in different layers. This means that you don't have to use the same aesthetics for all the geom functions.

```
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +
  # global mapping is now up here
  geom_point(mapping = aes(colour = Species, shape = Species)) +
  # local mappings to the points
  geom_smooth(mapping = aes(colour = Species, linetype = Species))
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



```
# local mappings to the line
```

5.2.6 Facetting your plots

The last element we will talk about now to add to your plot are facets because they are really useful and not complex to do. They are just another layer to add to the plot if you have categorical variable. Facets basically makes sub-plots that each display one subset of your data.

The first argument of `facet_wrap()` should be a formula, which you create with `~` followed by a discrete variable name.

```
# facet wrap by species
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +
  geom_point(mapping = aes(colour = Species, shape = Species)) +
  geom_smooth(mapping = aes(colour = Species)) +
  facet_wrap(~ Species, nrow = 1) # nrow is how many rows you want.
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



Now that it is faceted the species legend isn't really helpful. So let's remove it.

```
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +
  geom_point(mapping = aes(colour = Species, shape = Species), show.legend = FALSE) +
  geom_smooth(mapping = aes(colour = Species), show.legend = FALSE) +
  # note you need to remove from all mappings
  facet_wrap(~ Species, nrow = 1)
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



5.2.7 Changing axis ticks

Looking at the plot I would say it's a bit annoying that the ticks on the y axis (the numbers) are only every 2, I think they should be every 1 for consistency with the x.

To do this we just add another layer with the +

```
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +
  geom_point(mapping = aes(colour = Species, shape = Species), show.legend = FALSE) +
  geom_smooth(mapping = aes(colour = Species), show.legend = FALSE) +
  facet_wrap(~ Species, nrow = 1) +
  coord_cartesian(ylim=c(1, 7)) + # this zooms into the space between 1-7
  scale_y_continuous(breaks=seq(1, 7, 1)) # Ticks from 1-7, every 1
```

'geom_smooth()' using method = 'loess' and formula = 'y ~ x'



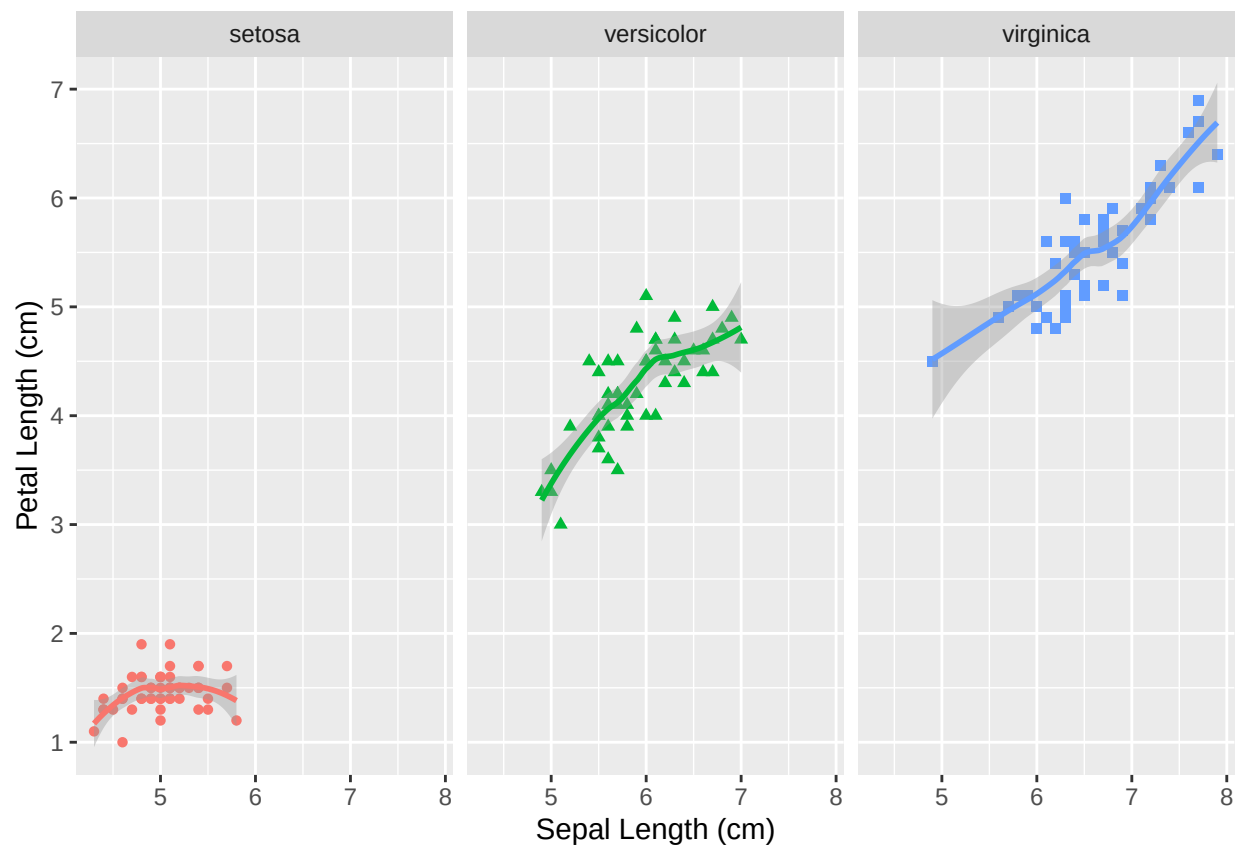
5.2.8 Text on plots

Finally, I think it would be nice if we didn't have the . between Sepal and length and Petal and length. So we need to change the text on the axes.

Again you just need to add the `ylab` (label for the y axis) and `xlab` (label for the x axis) functions to the existing plot.

```
# Change the axis labels
ggplot(data = iris, mapping = aes(x = Sepal.Length, y = Petal.Length)) +
  geom_point(mapping = aes(colour = Species, shape = Species), show.legend = FALSE) +
  geom_smooth(mapping = aes(colour = Species), show.legend = FALSE) +
  facet_wrap(~ Species, nrow = 1) +
  coord_cartesian(ylim=c(1, 7)) +
  scale_y_continuous(breaks=seq(1, 7, 1)) +
  ylab("Petal Length (cm)") + # rename the y axis
  xlab("Sepal Length (cm)") # rename the x axis
```

```
## 'geom_smooth()' using method = 'loess' and formula = 'y ~ x'
```



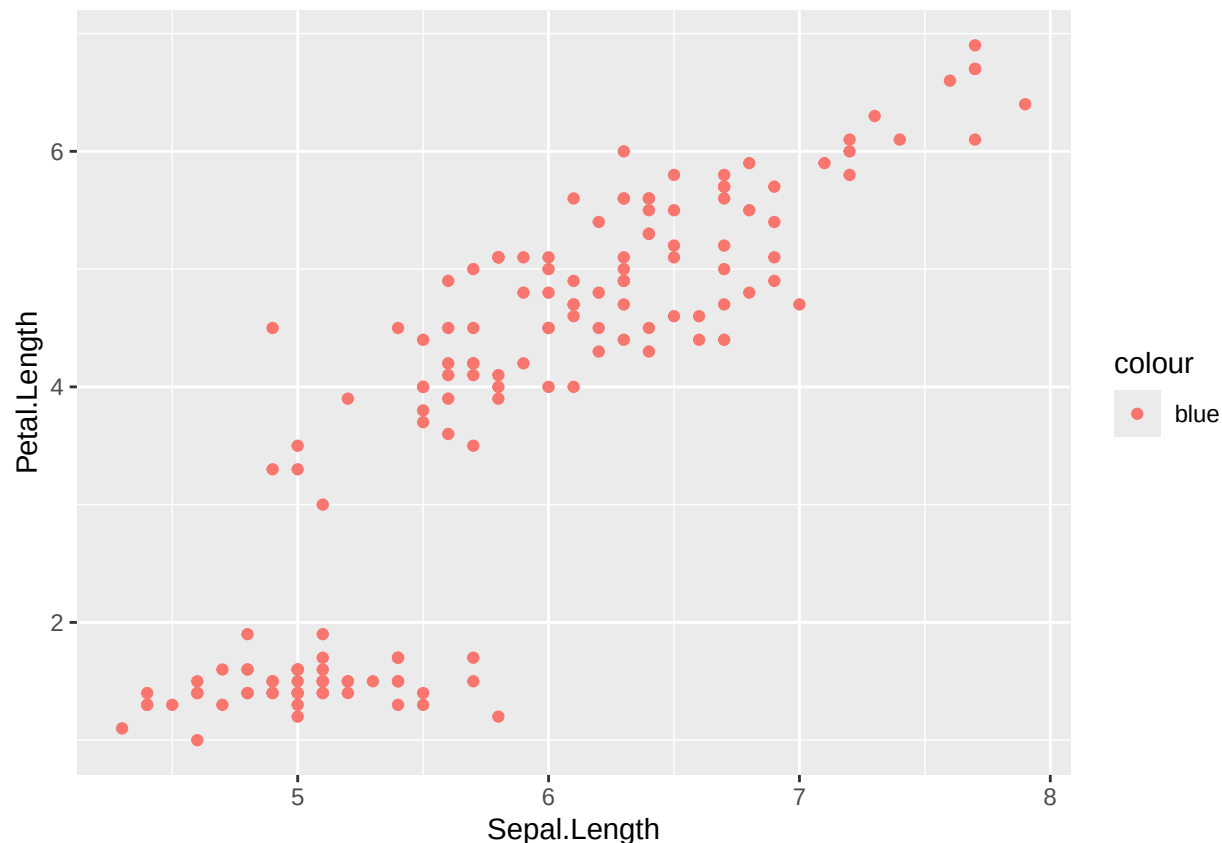
I'm pretty happy with this! This is how we build graphs - we start simple and then build in elements to best visualise the data.

5.2.9 Exercises 5.1:

A) Plot the relationship between sepal length and petal length (as above) but this time map petal width on to colour. How do the aesthetics behaviour differently for a continuous variable (like width)? Write a one sentence summary of the plot.

B) What is wrong with this code? Why are the points not blue?

```
ggplot(data = iris) +
  geom_point(mapping = aes(x = Sepal.Length, y = Petal.Length, color = "blue"))
```



C) Make a scatterplot between Petal.Length and Species. Does this still look like a Scatterplot. Try `geom_boxplot`.

D) Make a scatter plot between sepal and petal width. Add in aesthetics to better visualise the data. Write a one sentence summary of the plot.

5.2.9.1 Problem shooting It is really common to run a line of code and it doesn't work. Normally it is a misplaced character, or an unclosed bracket. Check that every (has a) and every " has a ".

Capitals also matter in R - note that `Sepal.Length` will work but `sepal.length` will not!

Sometimes you will run the code and nothing happens. Check the left-hand of your console: if its a `+`, it means that R doesn't think you have typed a complete expression and its waiting for you to finish it. In this case, its usually easy to start from scratch again by pressing `ESCAPE` to abort processing the current command.

One common problem when creating `ggplot2` graphics is to put the `+` in the wrong place: it has to come at the end of the line, not the start.

If you're still stuck, try the help. You can get help about any R function by running `?function_name` in the console, or selecting the function name and pressing `F1` in RStudio. Don't worry if the help doesn't seem that helpful - instead skip down to the examples and look for code that matches what you're trying to do.

If that doesn't help, carefully read the error message. Sometimes the answer will be buried there! But when you're new to R, the answer might be in the error message but you don't yet know how to understand it. Another great tool is Google: try googling the error message, as it's likely someone else has had the same problem, and has gotten help online.

5.2.9.2 Further resources for plotting See: Cheatsheet for ggplot2

See: Cookbook for R Graphics

6 Conditional Programming: Week 6

6.1 Logical values

6.2 if and else statements

7 Pipes and Loops: Week 7

7.1 Pipes (`%>%`)

7.2 for loops

7.3 Repeating operations

8 Writing Functions from scratch: Week 8

8.1 Why write functions?

8.2 Function structure

8.3 Writing your first function

9 Simulating Data: Week 9

9.1 Why simulate data?

9.2 Generating random data

9.3 A simple simulation example